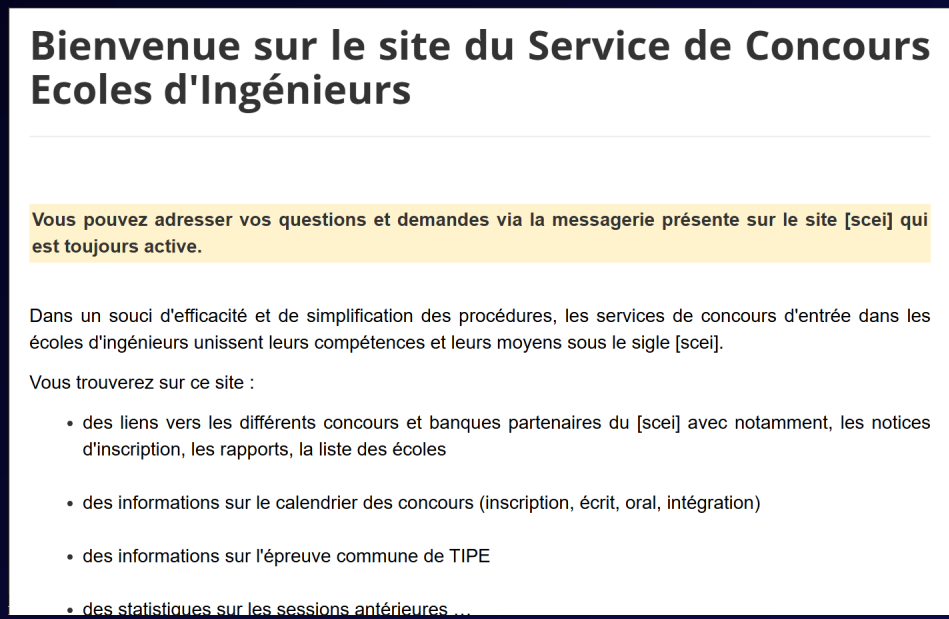
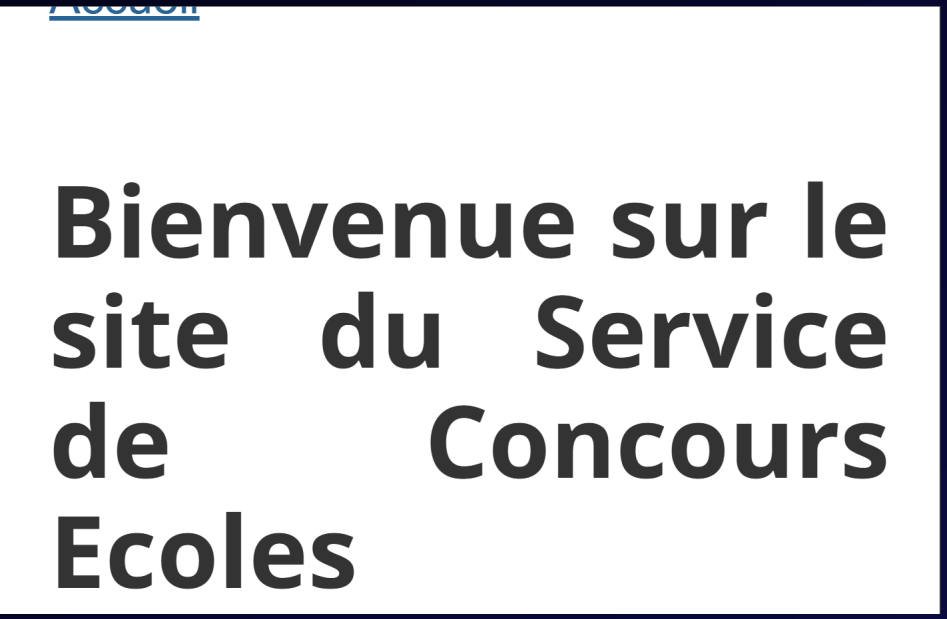
Compression d’images vectorielles

AUBERT--MÉZIÈRE Corentin
n° 27653

Exemples images vectorielles :

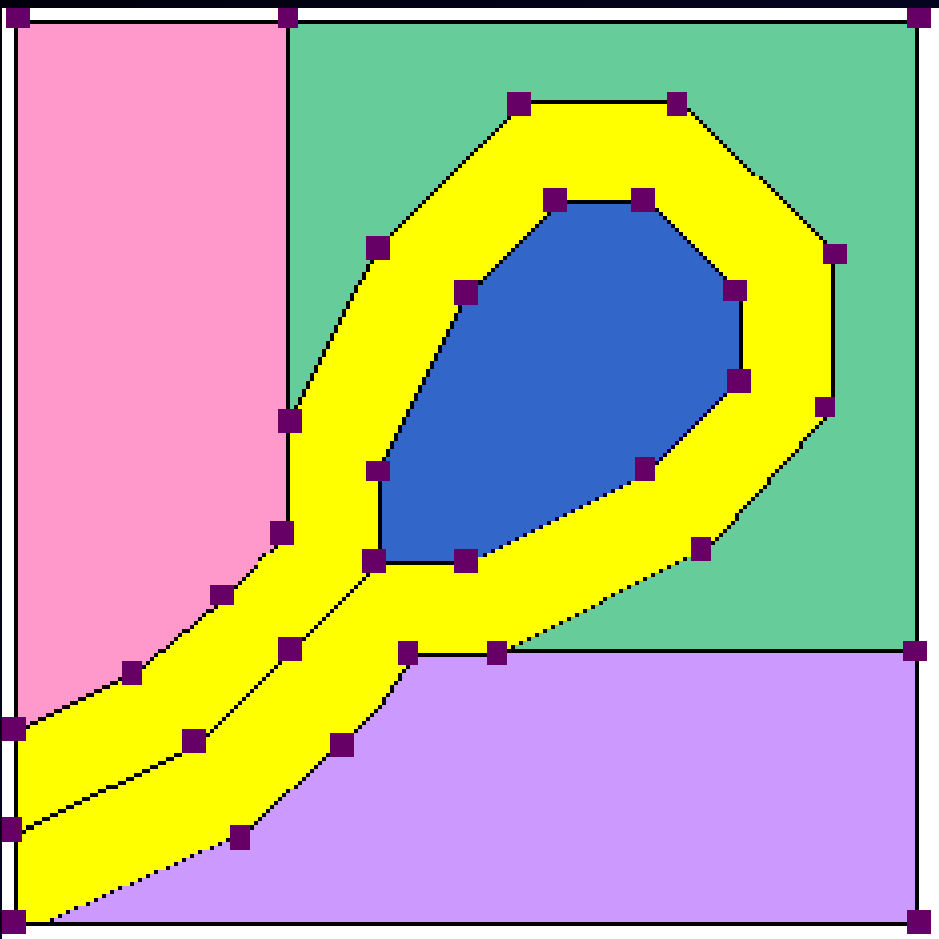




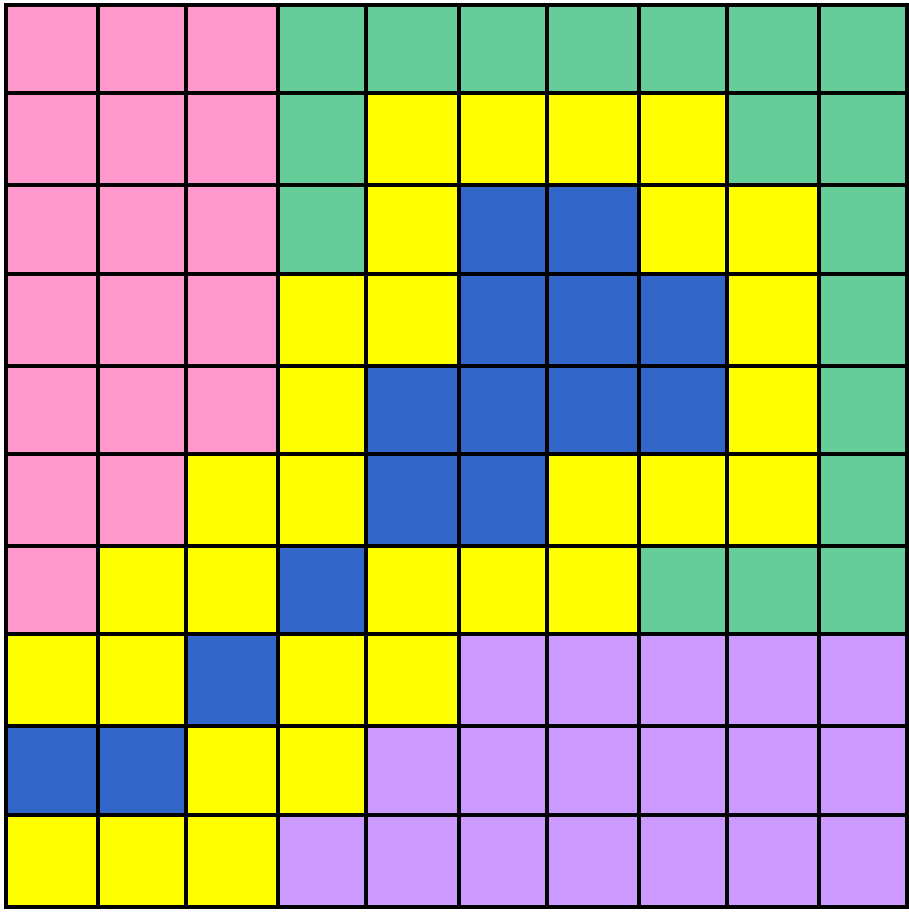




Exemples images vectorielles :



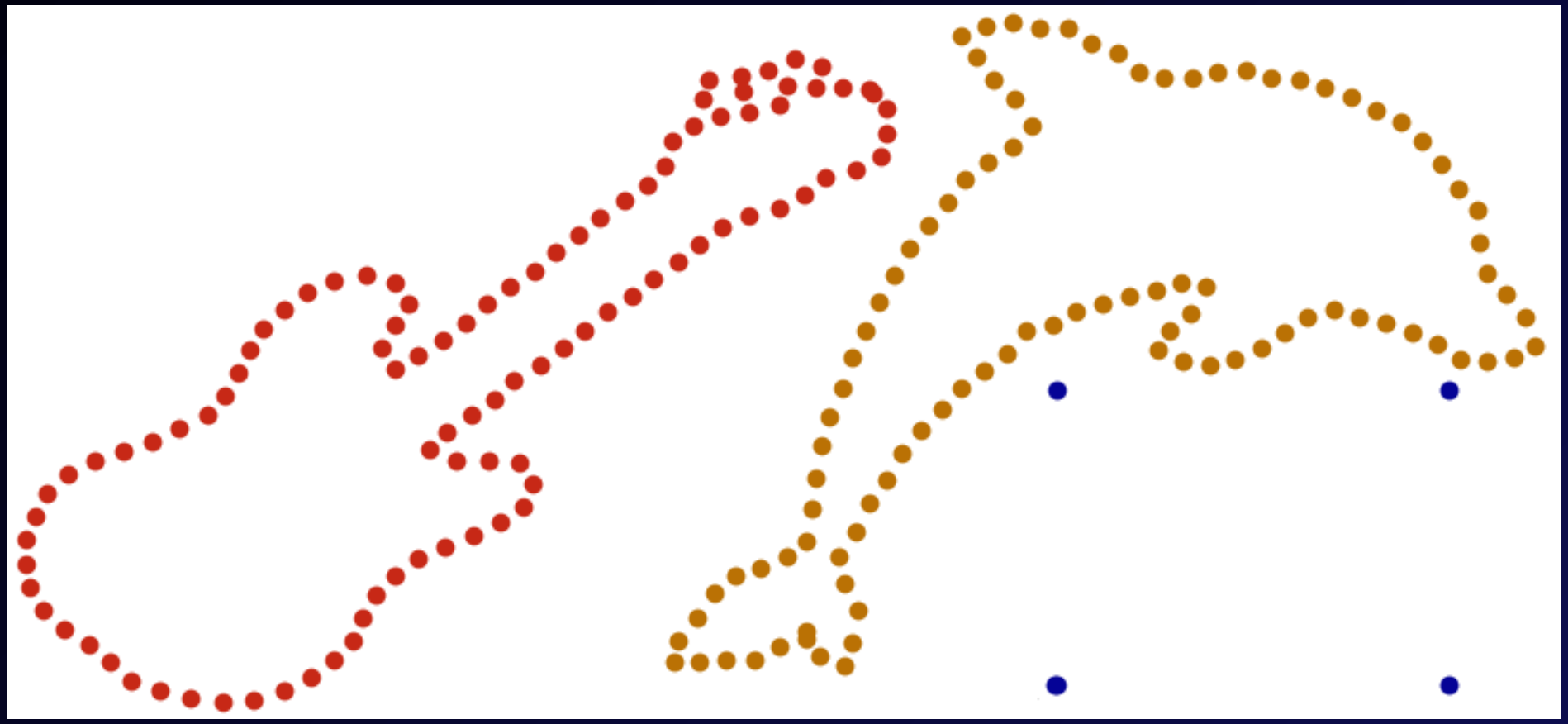
Vector



Raster

Source : Department of Geography, The Pennsylvania State University (psu.edu)

Exemples images vectorielles :



Méthodes utilisées :

Transformée de Fourier
Transformée de Fourier Discrète (TFD)
Courbes de Bézier + Algorithme de Schneider

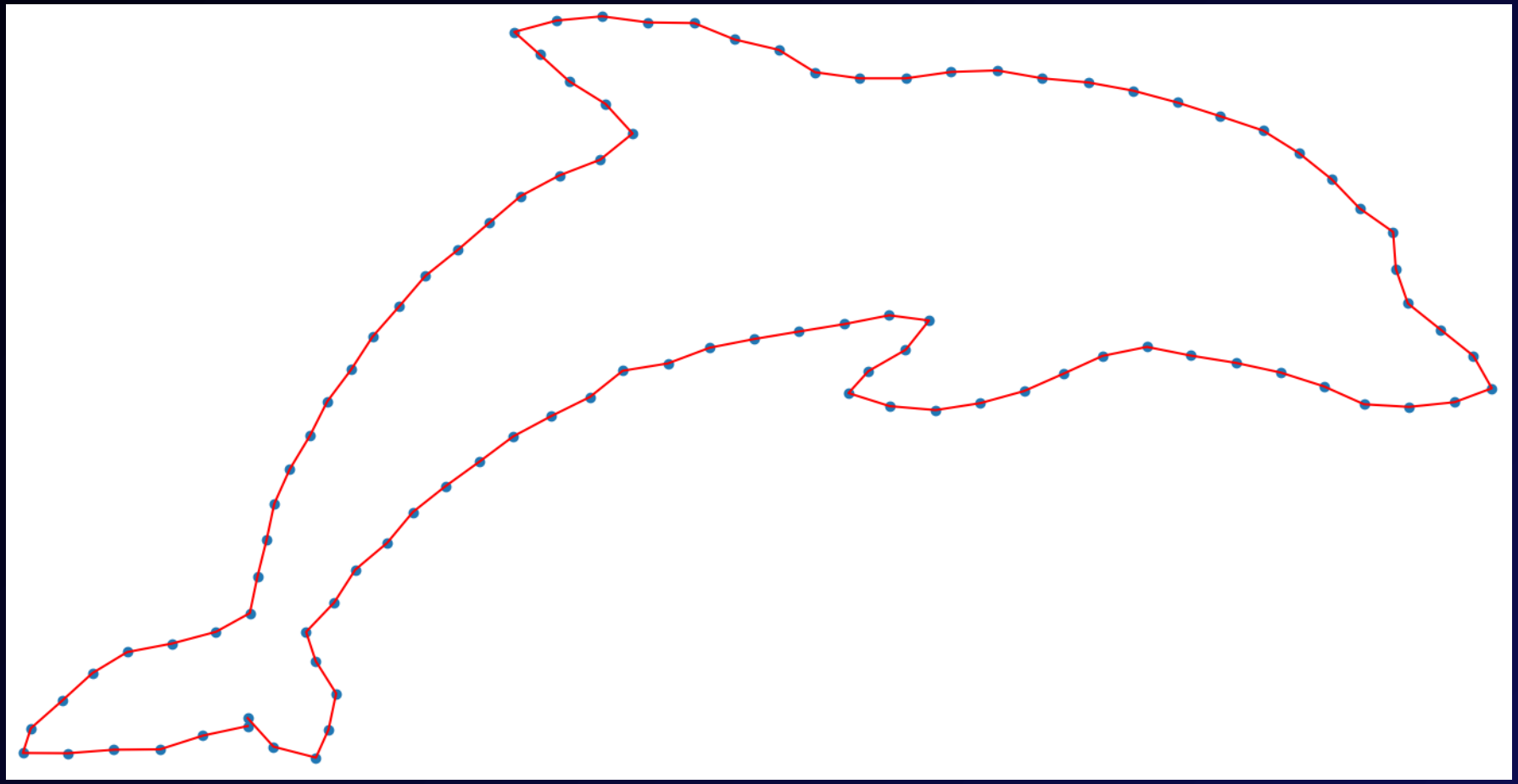
Soit $f : \mathbb{R} \to \mathbb{C}$, T-périodique

Coefficients de Fourier complexes :

$$\forall k \in \mathbb{Z},$$

$$c_k(f) = \frac{1}{T} \int_0^T f(t) e^{-ik\frac{2\pi}{T}t} \, dt$$

$f : \mathbb{R} \to \mathbb{C}$, $\mathcal{C}^1$, 1-périodique



On pose pour tout k entier : $c_k(f) = C(k)$

Théorème de Dirichlet :

Soit f une fonction continue, C1 par morceaux, périodique.
Alors la série de Fourier de f converge normalement vers f.

$$f(t) = \sum_{p=-\infty}^{+\infty} C(p) e^{2i\pi pt}$$

$$C(k) = \int_0^1 f(p) e^{-2i\pi kp} \, dp$$

$$f(t) = \sum_{p=-\infty}^{+\infty} C(p) e^{2i\pi pt}$$

$$C(p) = \frac{1}{N} \sum_{k=0}^{N-1} f\left(\frac{k}{N}\right) e^{-i2\pi p \frac{k}{N}}$$

$$f(t) = \sum_{p=-A}^{A} C(p) e^{2i\pi pt}$$

Taille fonction compressée : 2A + 1 (harmoniques)

Quantité de données économisées :

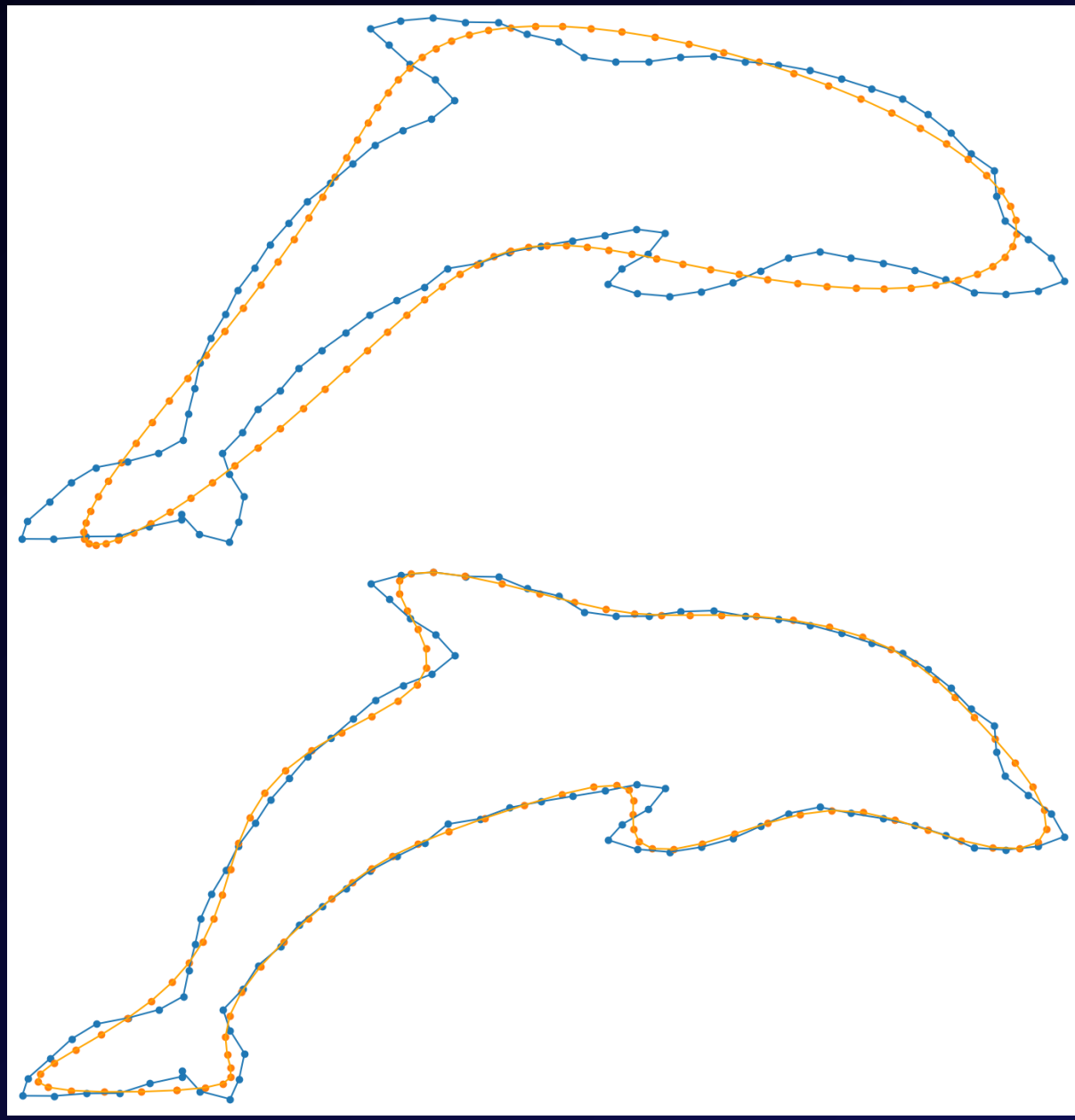
taux de compression : $\tau = 1 - \frac{taille\ obtenue}{taille\ initiale}$

A = 3

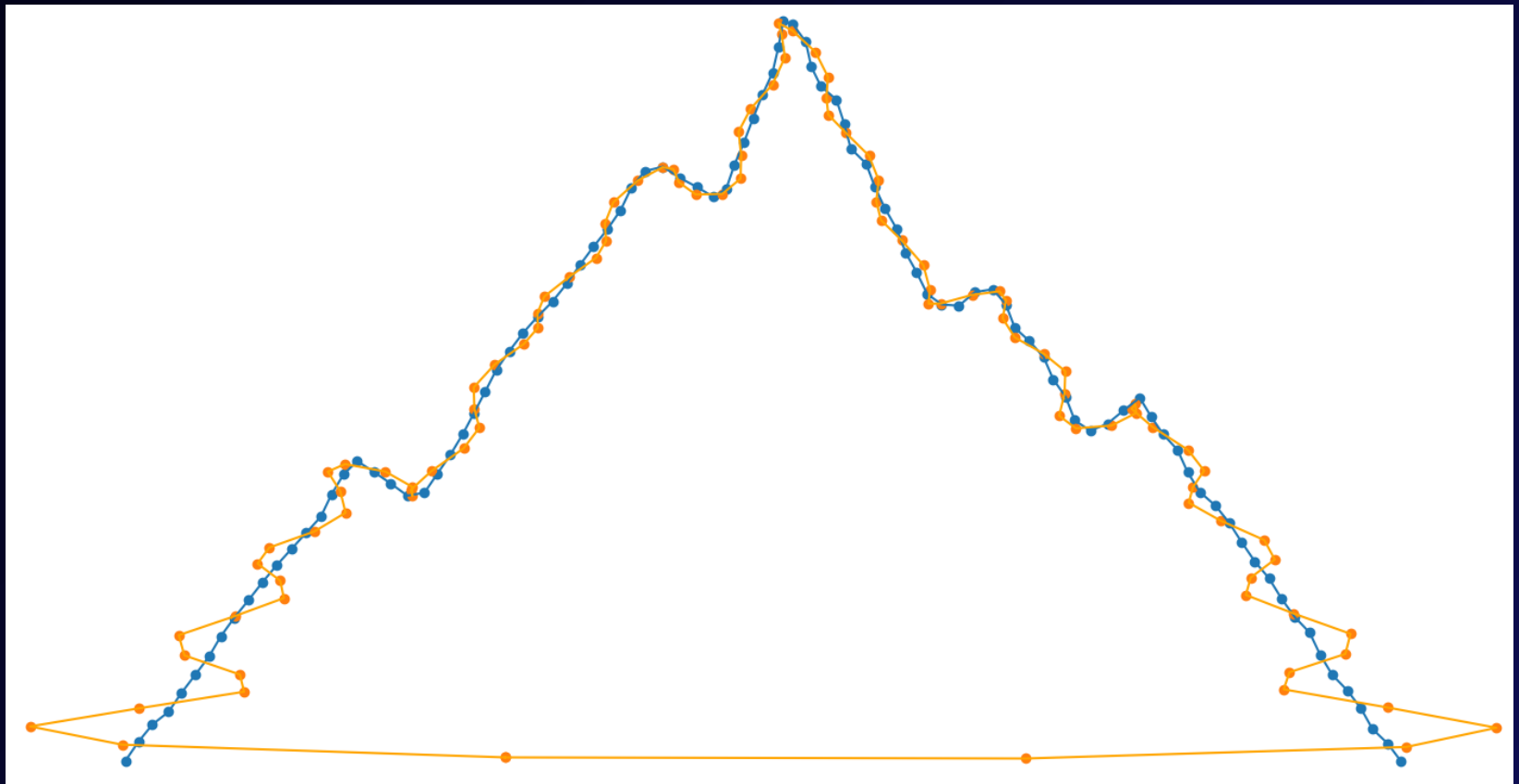
τ = 93 %

A = 10

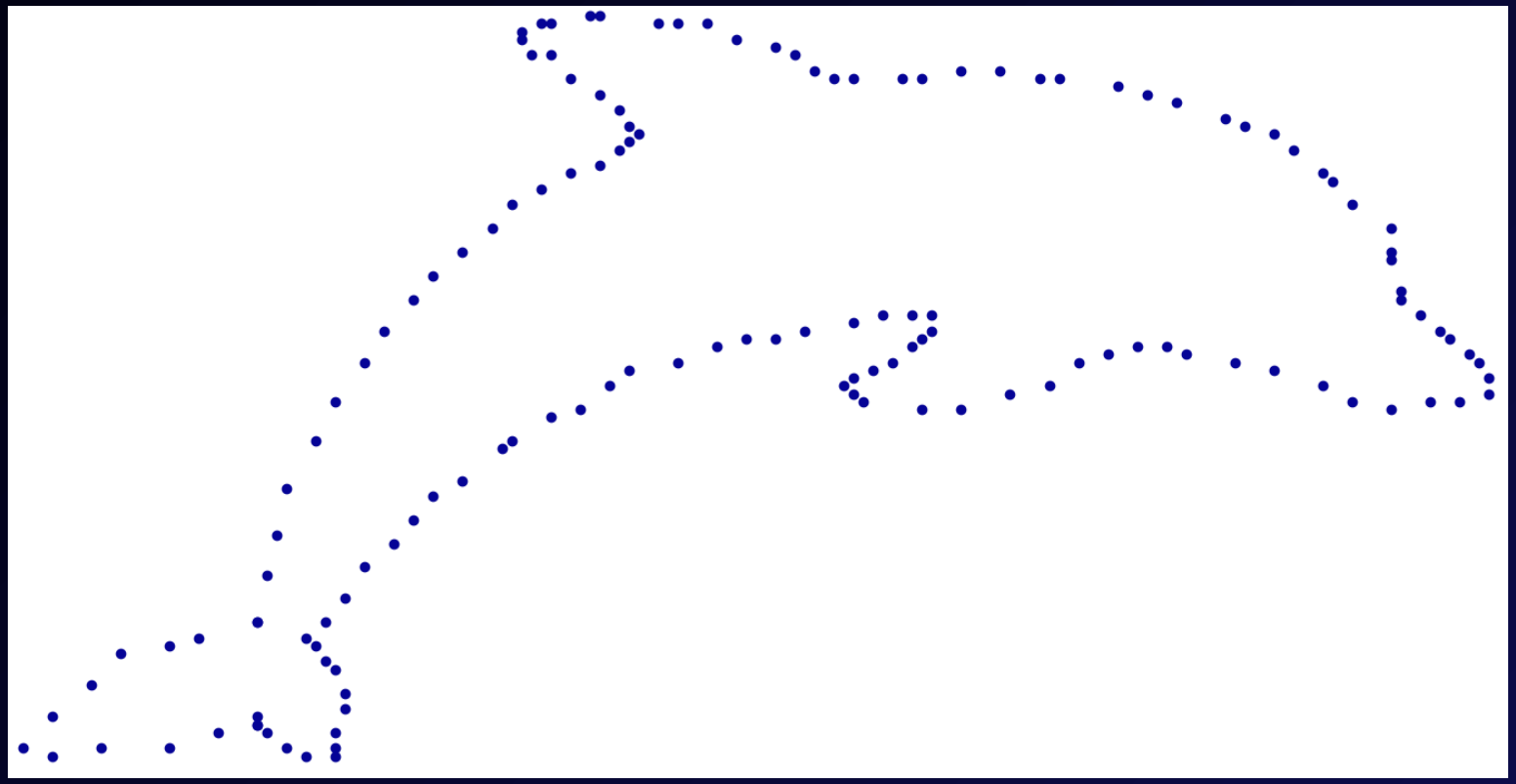
τ = 79 %



Théorème de Dirichlet :
Soit f une fonction C1 par morceaux, périodique. Alors pour tout réel x, la série de Fourier de f converge vers $\frac{f(x^+) + f(x^-)}{2}$



Coordonnées d'origine : $x(0), \ldots, x(N-1)$



Transformée de Fourier discrète de ce vecteur :

$$X(0), \ldots, X(N-1)$$

$$B = \begin{pmatrix} 1 & 1 & \cdots & 1 \\ 1 & \omega_N^{1.1} & \cdots & \omega_N^{1.(N-1)} \\ \vdots & \vdots & \ddots & \vdots \\ 1 & \omega_N^{(N-1).1} & \cdots & \omega_N^{(N-1)^2} \end{pmatrix} A$$

$$A = \begin{pmatrix} x(0) \\ x(1) \\ \vdots \\ x(N-1) \end{pmatrix}, B = \begin{pmatrix} X(0) \\ X(1) \\ \vdots \\ X(N-1) \end{pmatrix}, w_N = e^{-\frac{i2\pi}{N}}$$

$$\forall k \in [\![0, N-1]\!],$$

$$X(k) = \sum_{n=0}^{N-1} x(n) e^{-i2\pi \frac{nk}{N}}$$

$$x(k) = \frac{1}{N} \sum_{n=0}^{N-1} X(n) e^{i2\pi \frac{nk}{N}}$$

Transformation de B en B’ :

Repérage des J plus grandes coordonnées (en valeur absolue/module ; J entier naturel)
Suppression des autres coordonnées

Résultat :

vecteur de J coordonnées non nulles

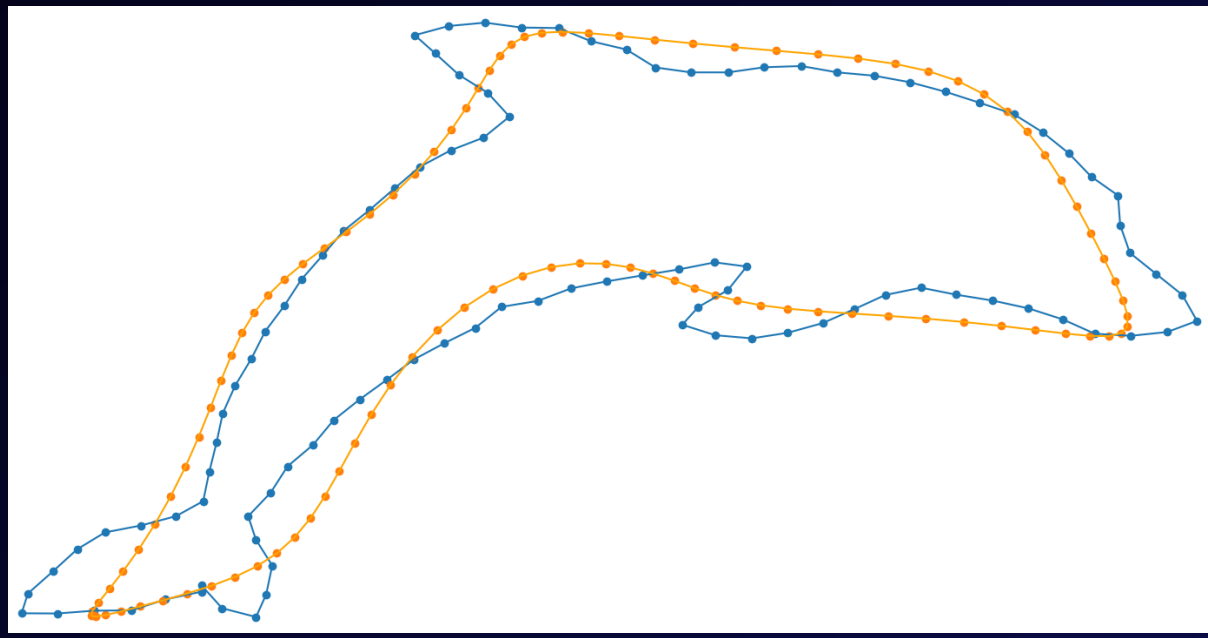
Taille vecteur compressé :

J coordonnées

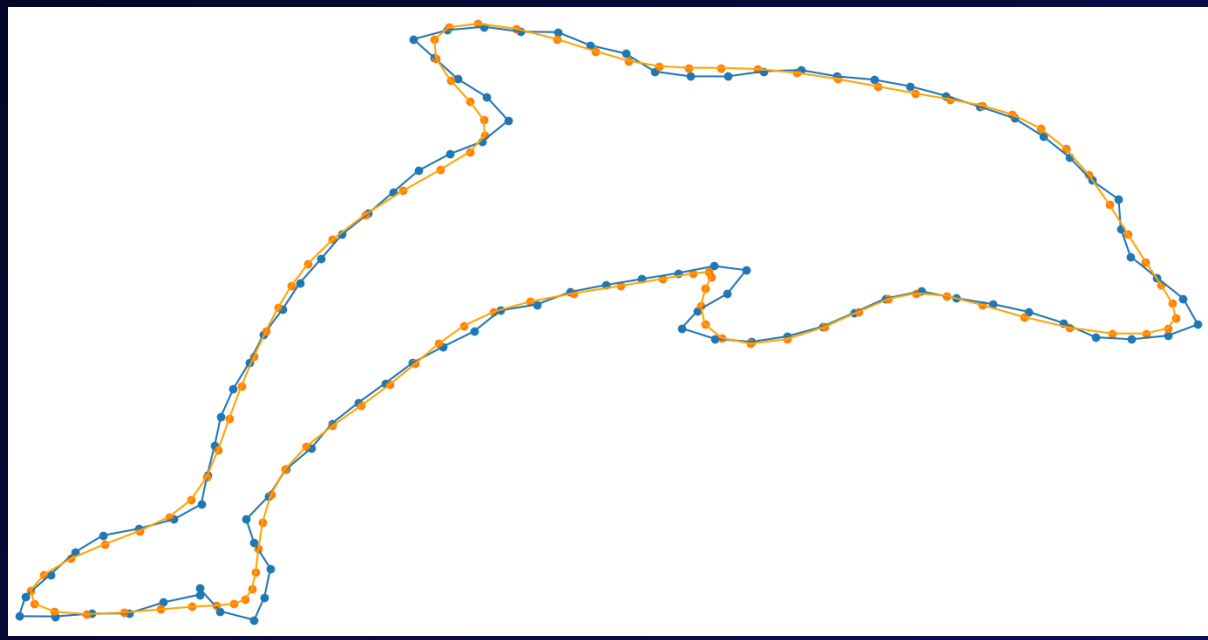
Décompression image :

Appliquer TFD inverse sur F

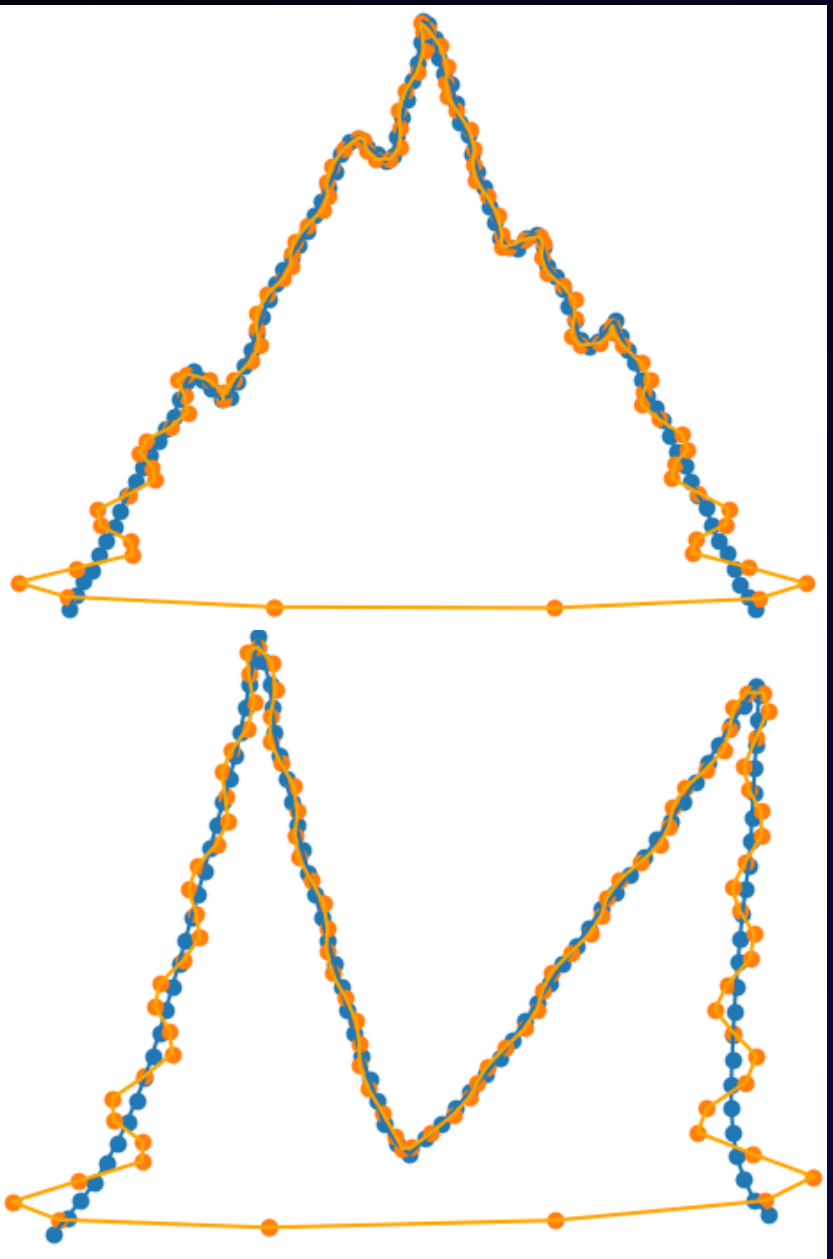
J = 7

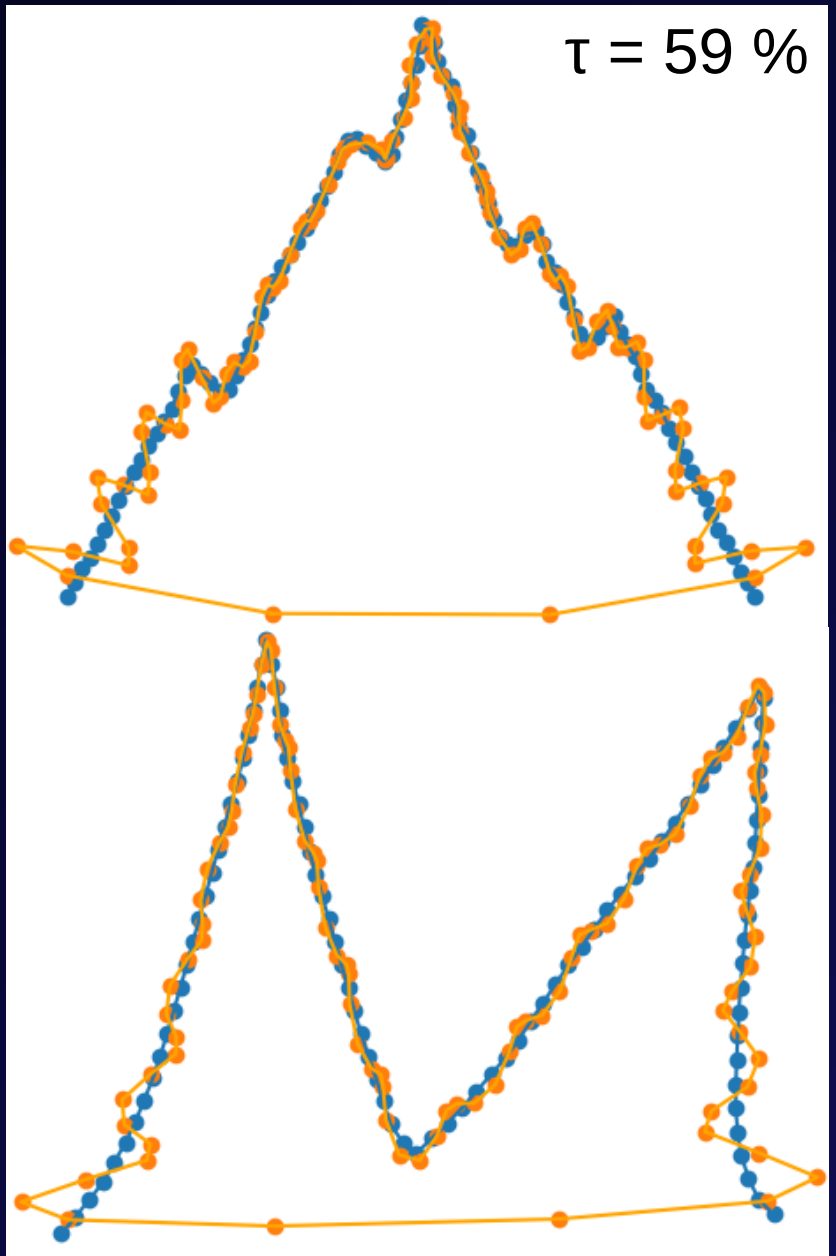


J = 21



Problème courbe non fermée :





Évaluation des algorithmes

Qualité de courbe obtenue :

$$(A, A') \in (\mathbb{C}^n)^2$$

distance = $||A - A'||_1$

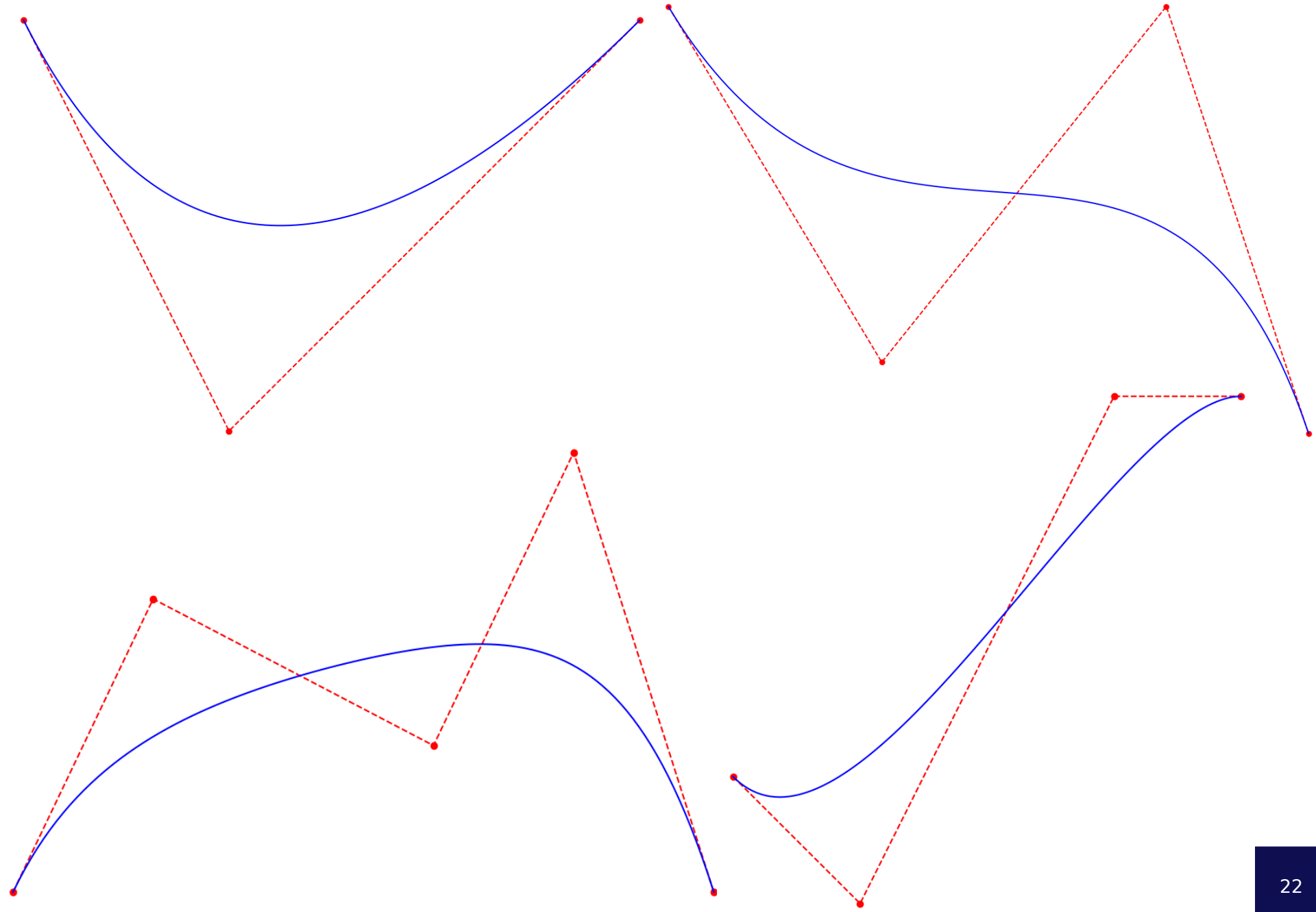
$$\forall x = (x_1, \ldots, x_n) \in \mathbb{C}^n, \|x\|_1 = \sum_{i=1}^{n} |x_i|$$

Courbes de Bézier :

Points de contrôle $P_0, \ldots, P_n$

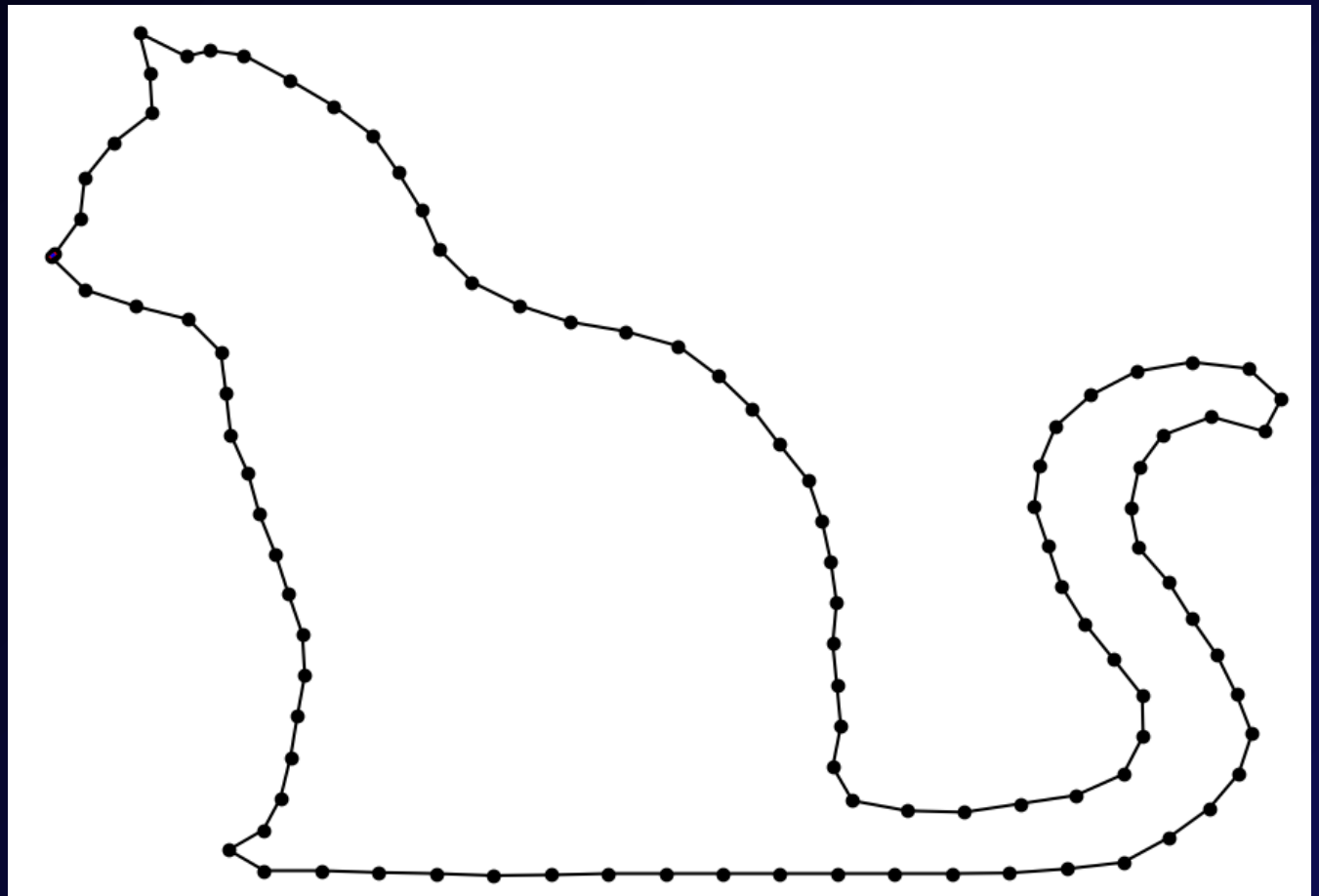
$$\forall t \in [0,1], P(t) = \sum_{i=0}^{n} B_i^n(t) P_i$$

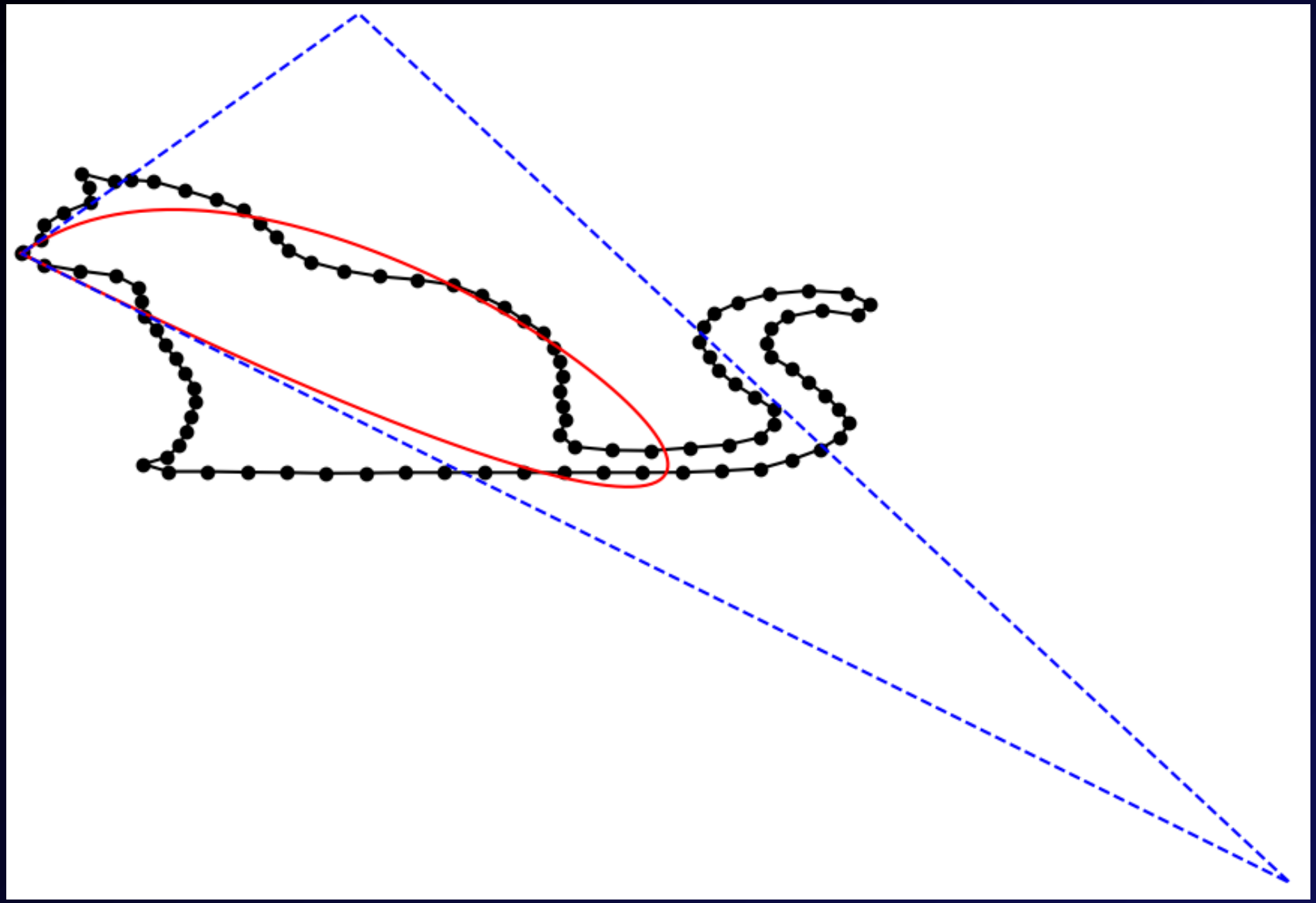
$$B_i^n(t) = \binom{n}{i} t^i (1-t)^{n-i}$$

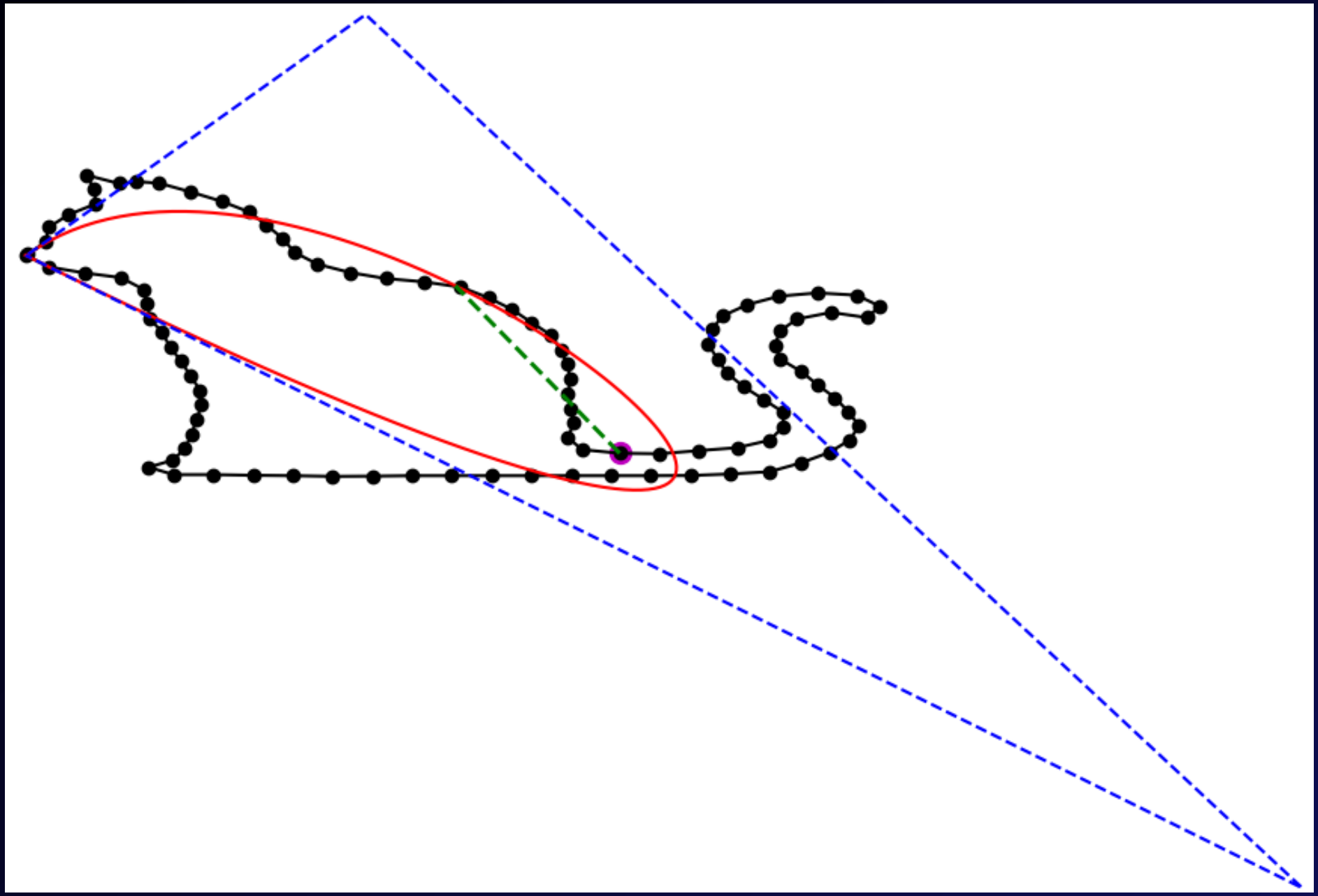


Algorithme de Schneider :

Objectif : Créer et ajuster une succession de Béziers cubiques à la courbe d'origine

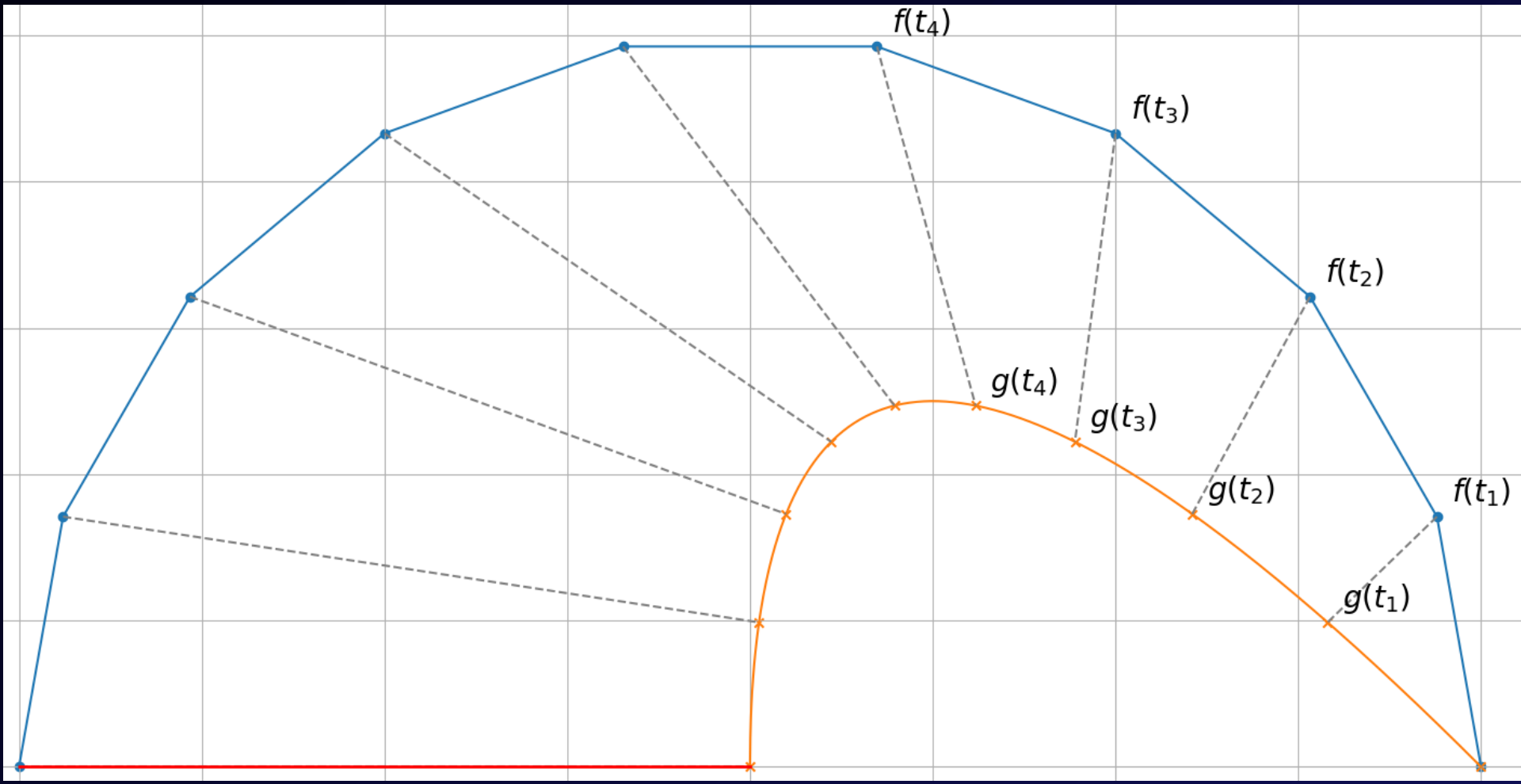


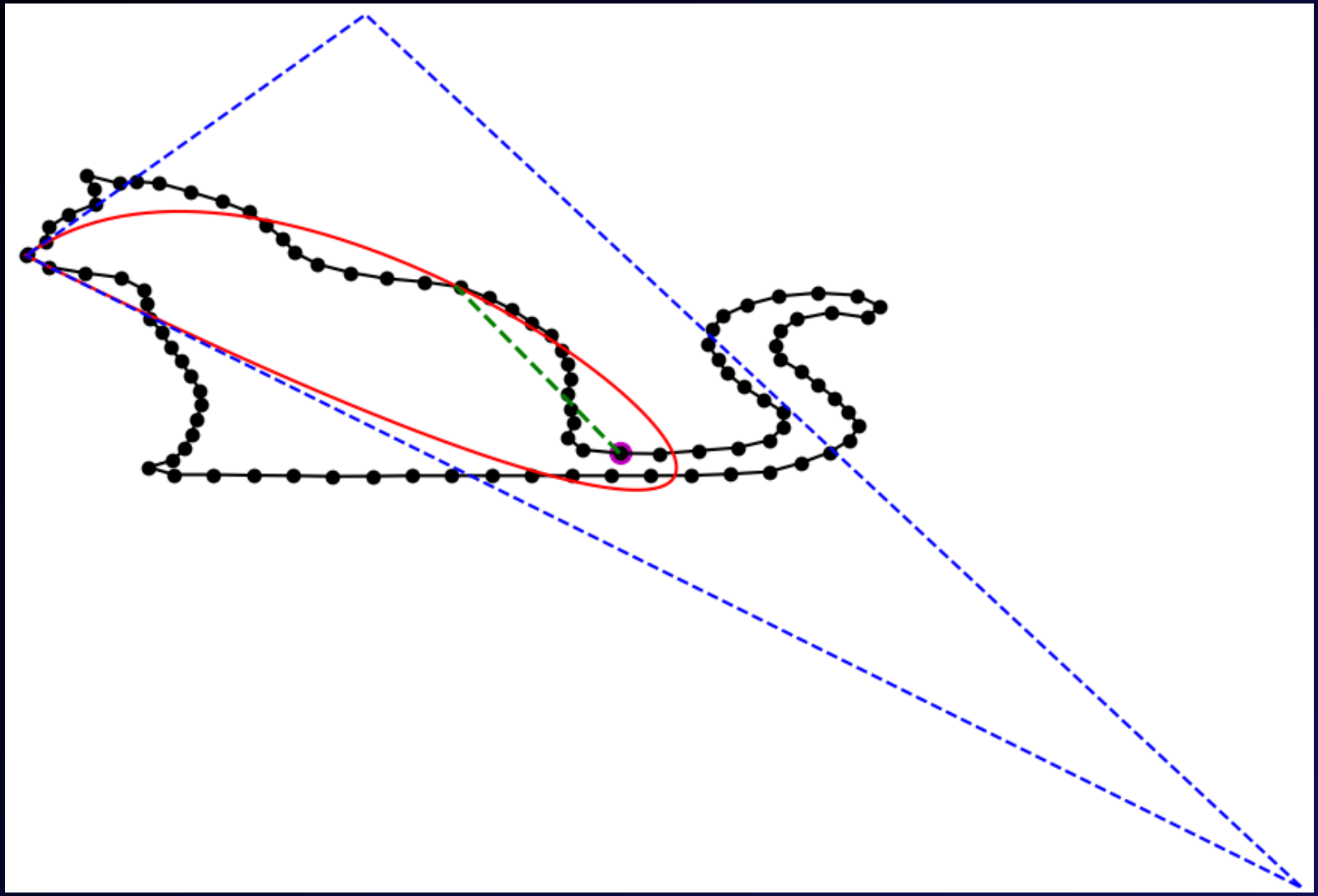


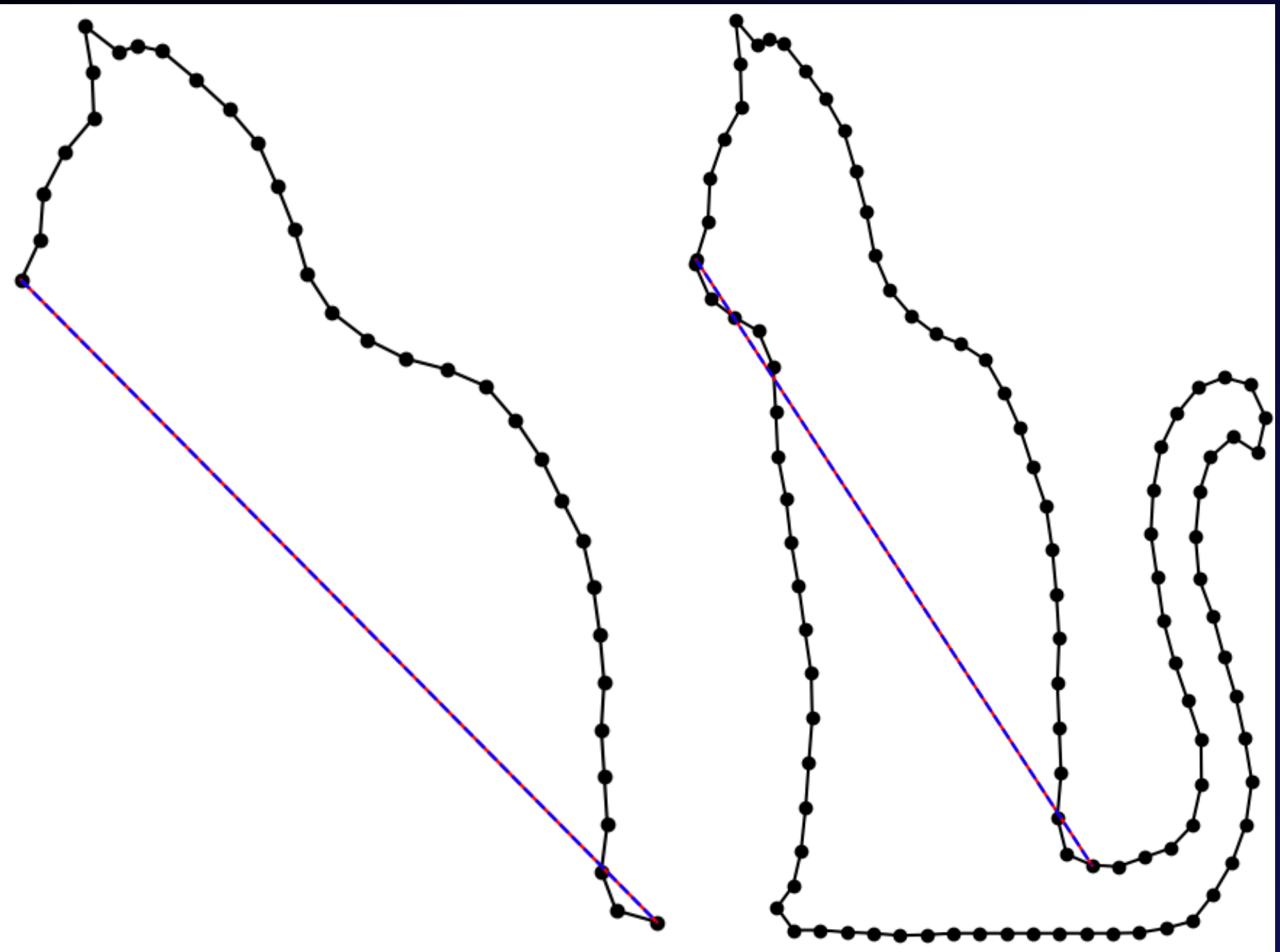


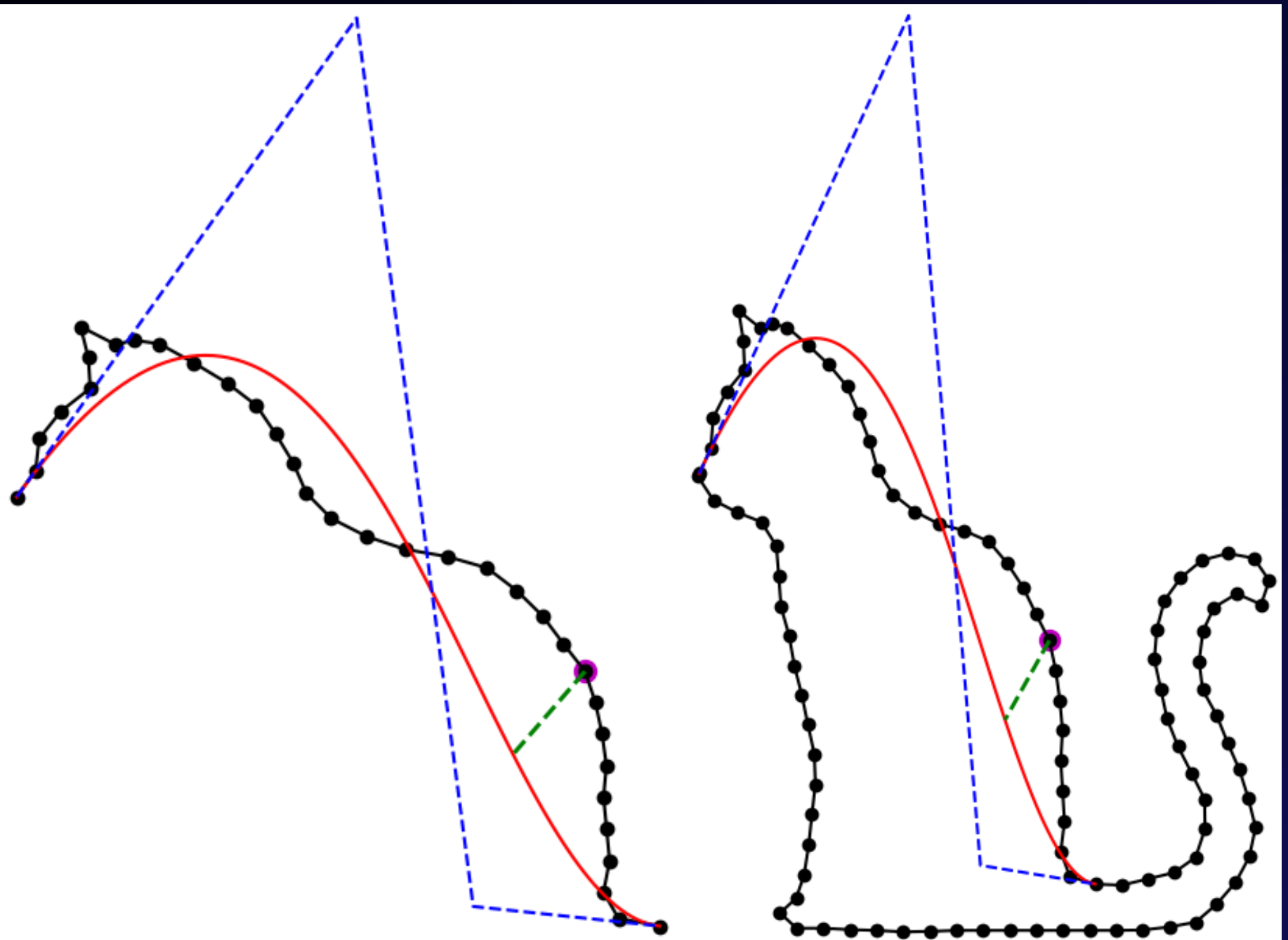
Distance utilisée dans l'algorithme de Schneider :

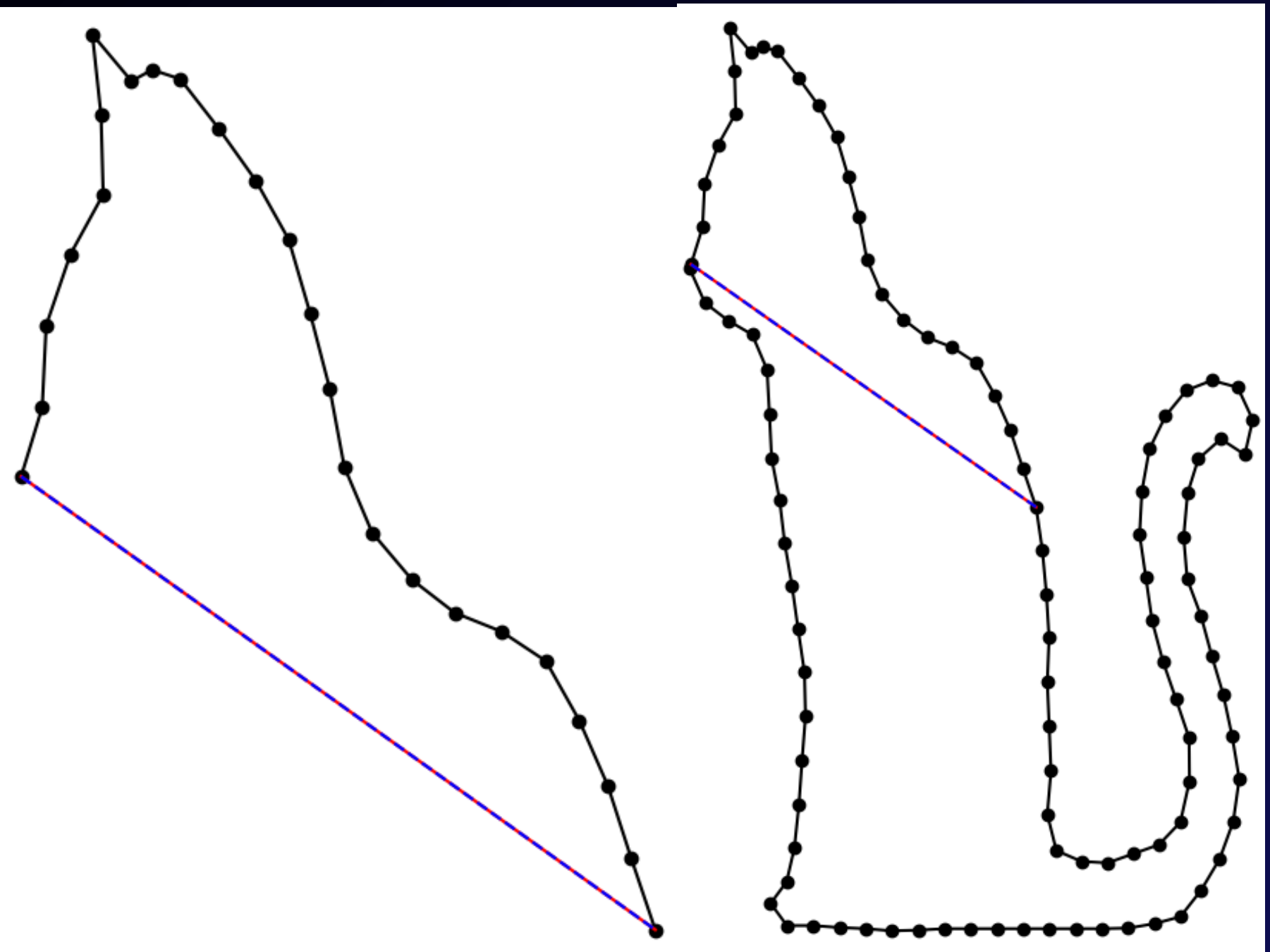
$$d = \max_{t \in \{t_0, \ldots, t_n\}} |f(t) - g(t)|$$

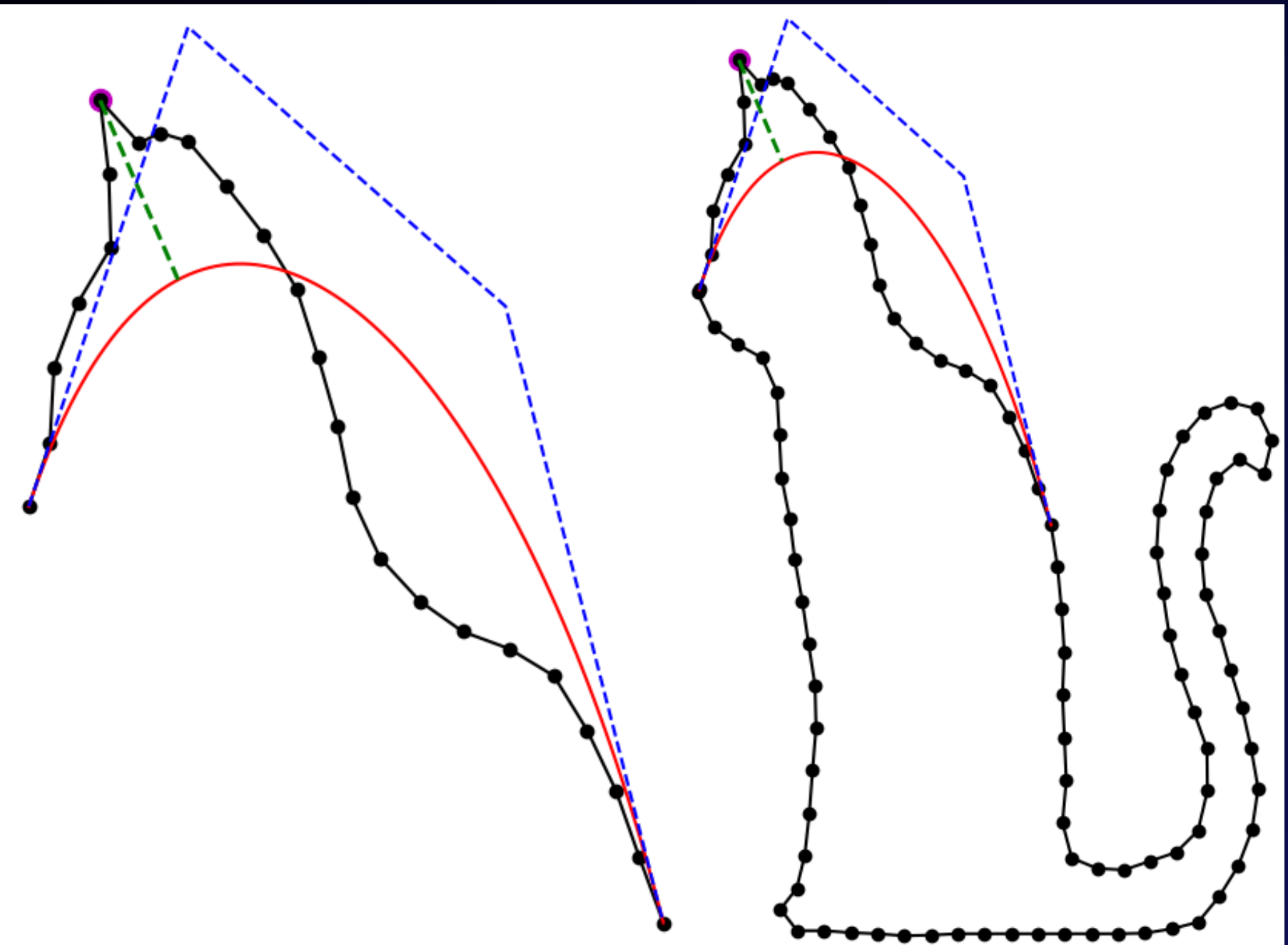


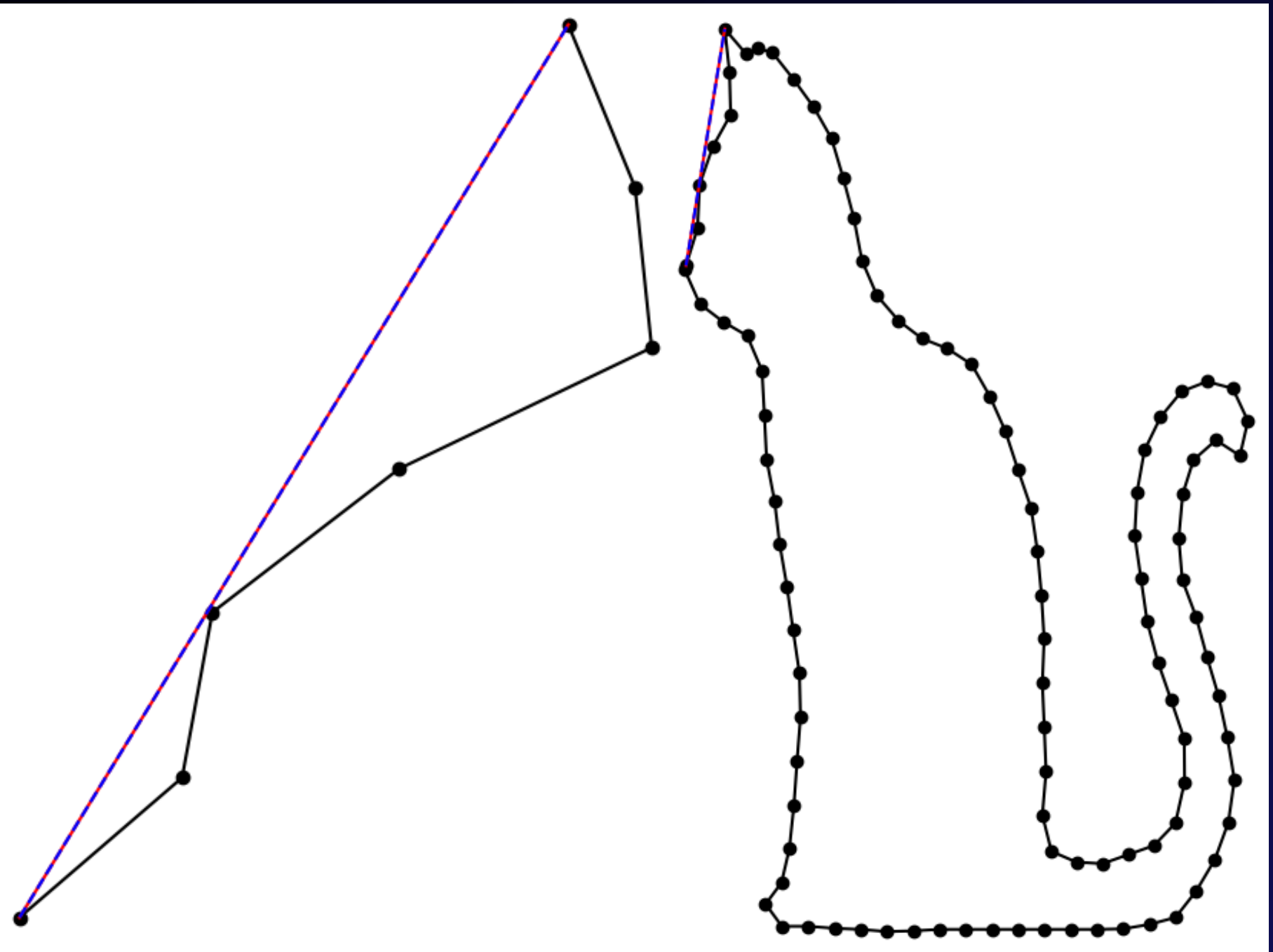


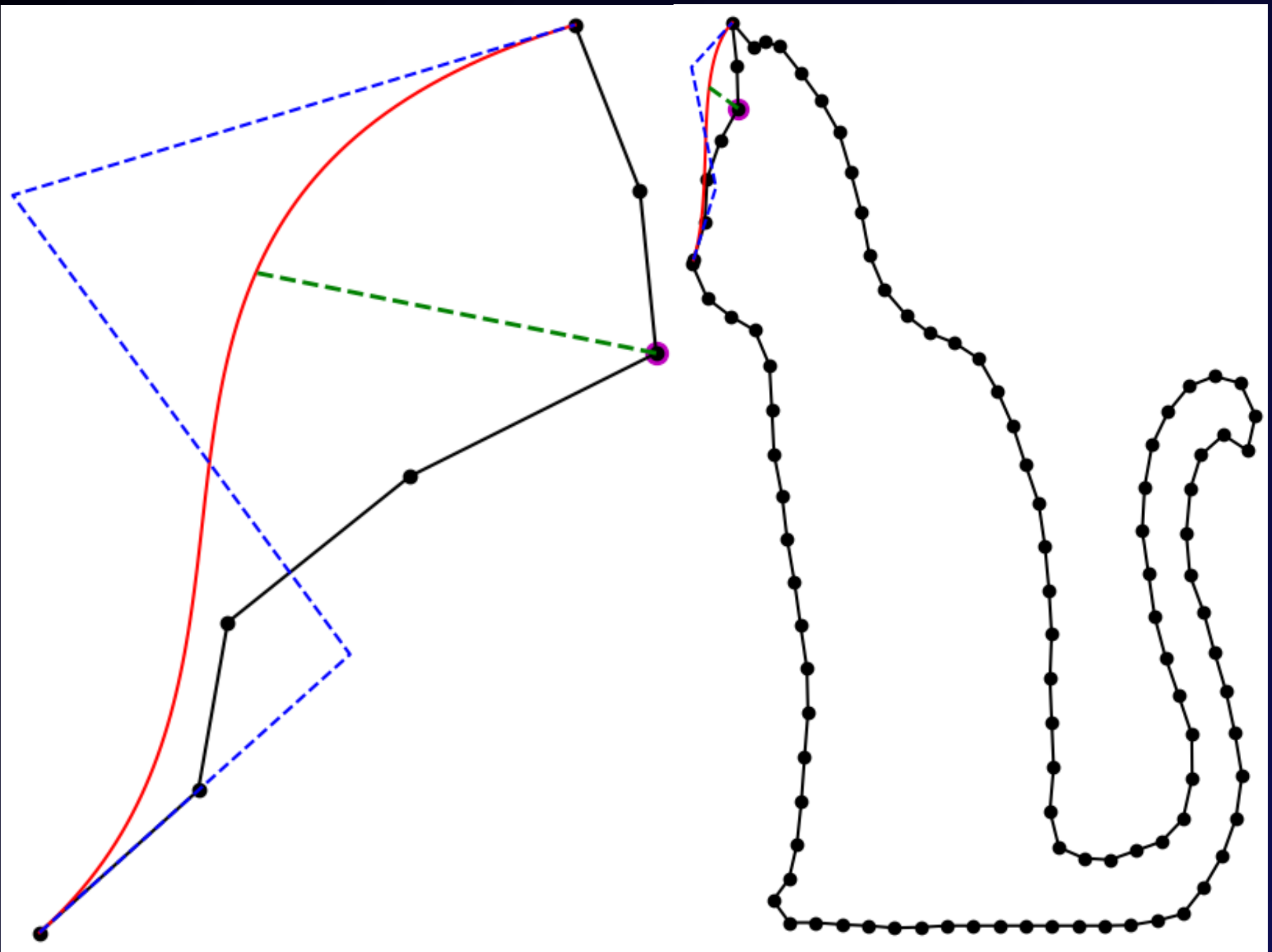


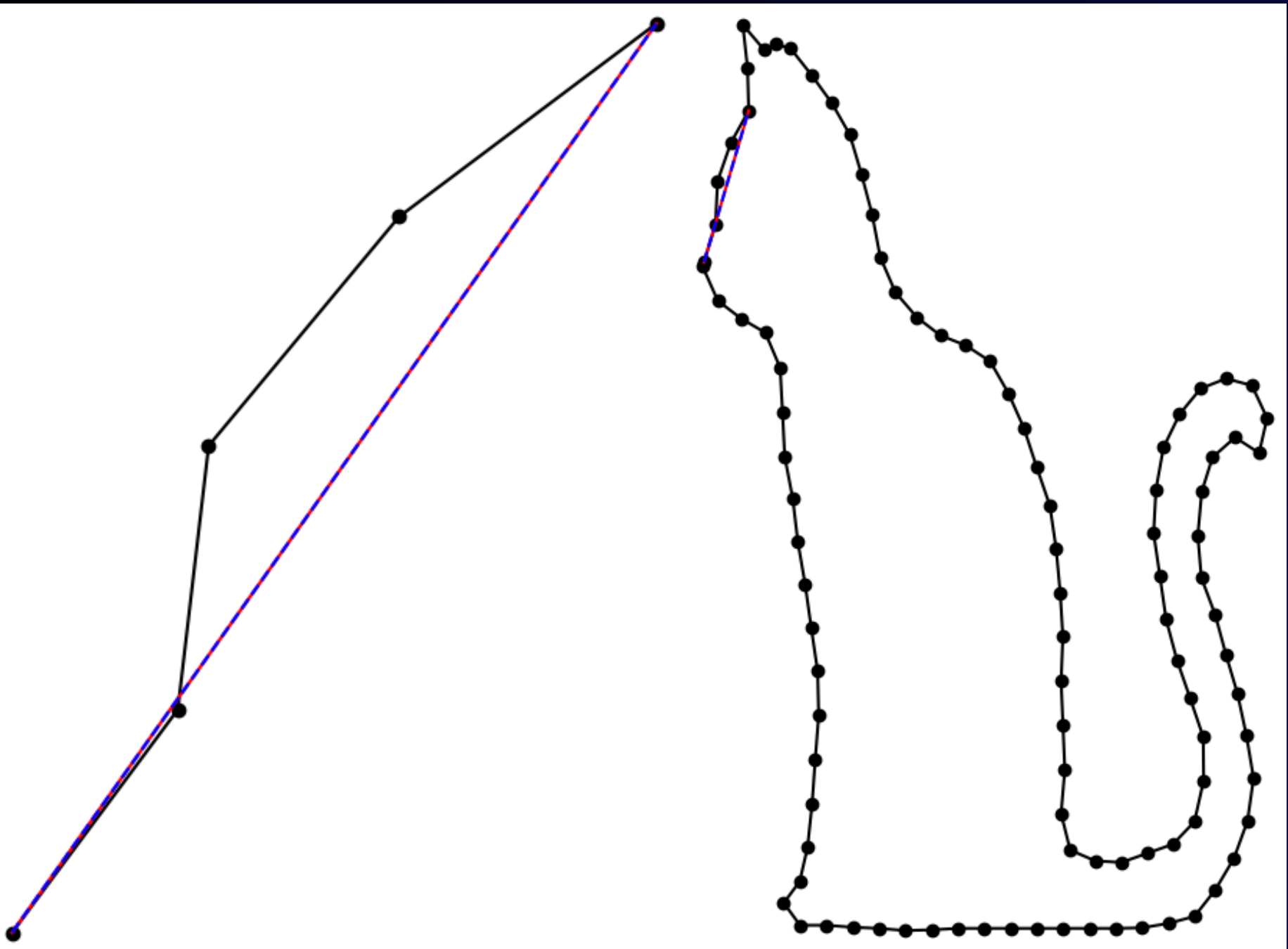


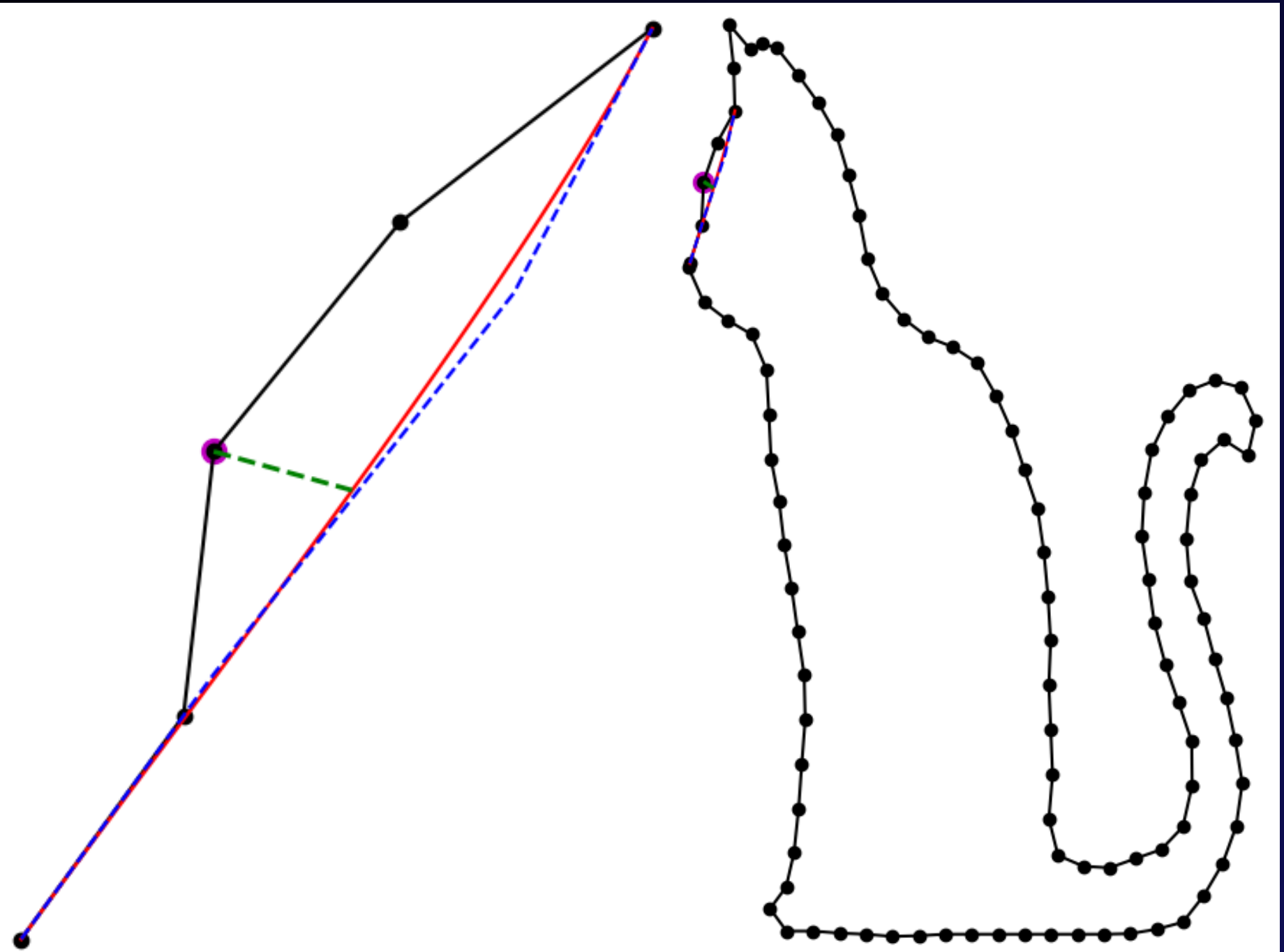


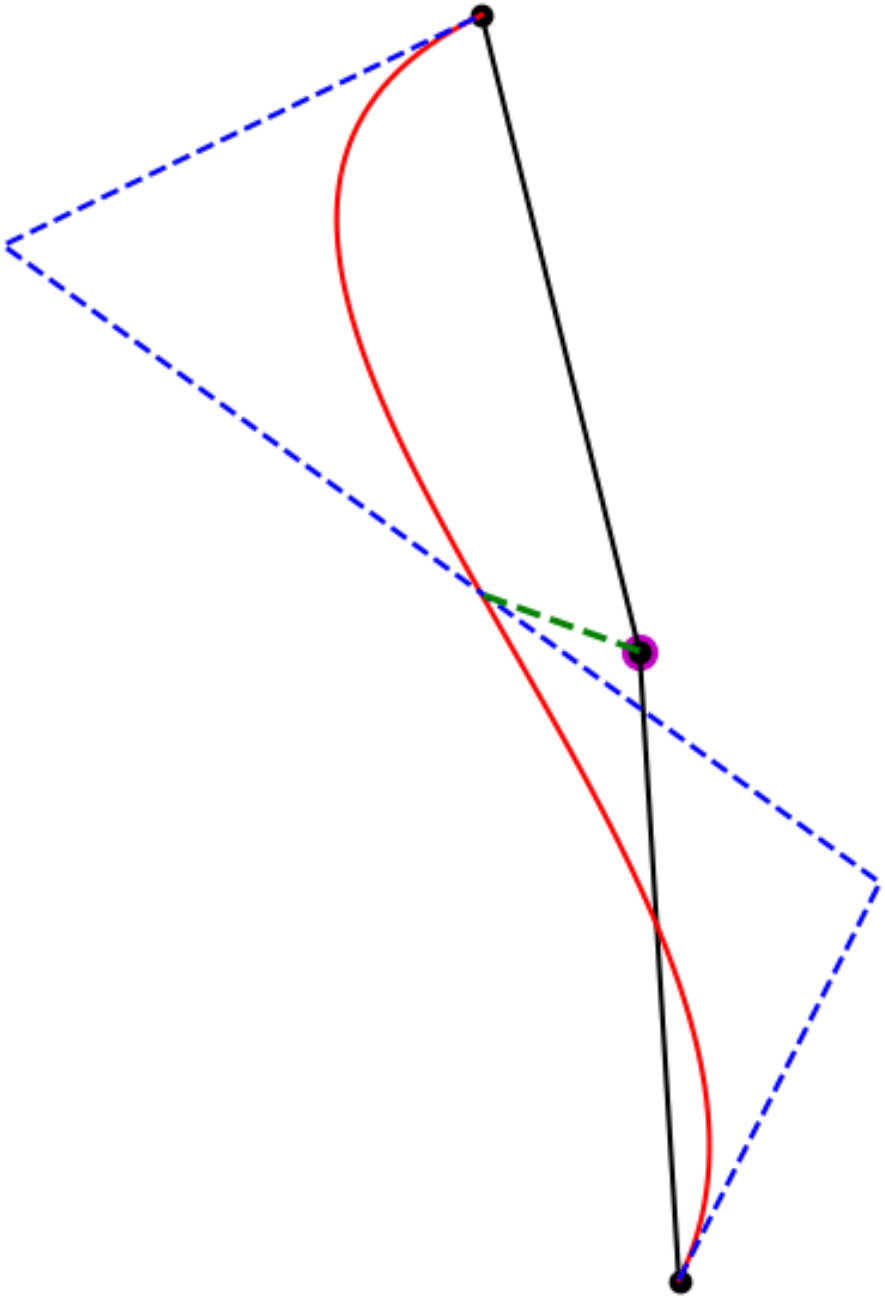


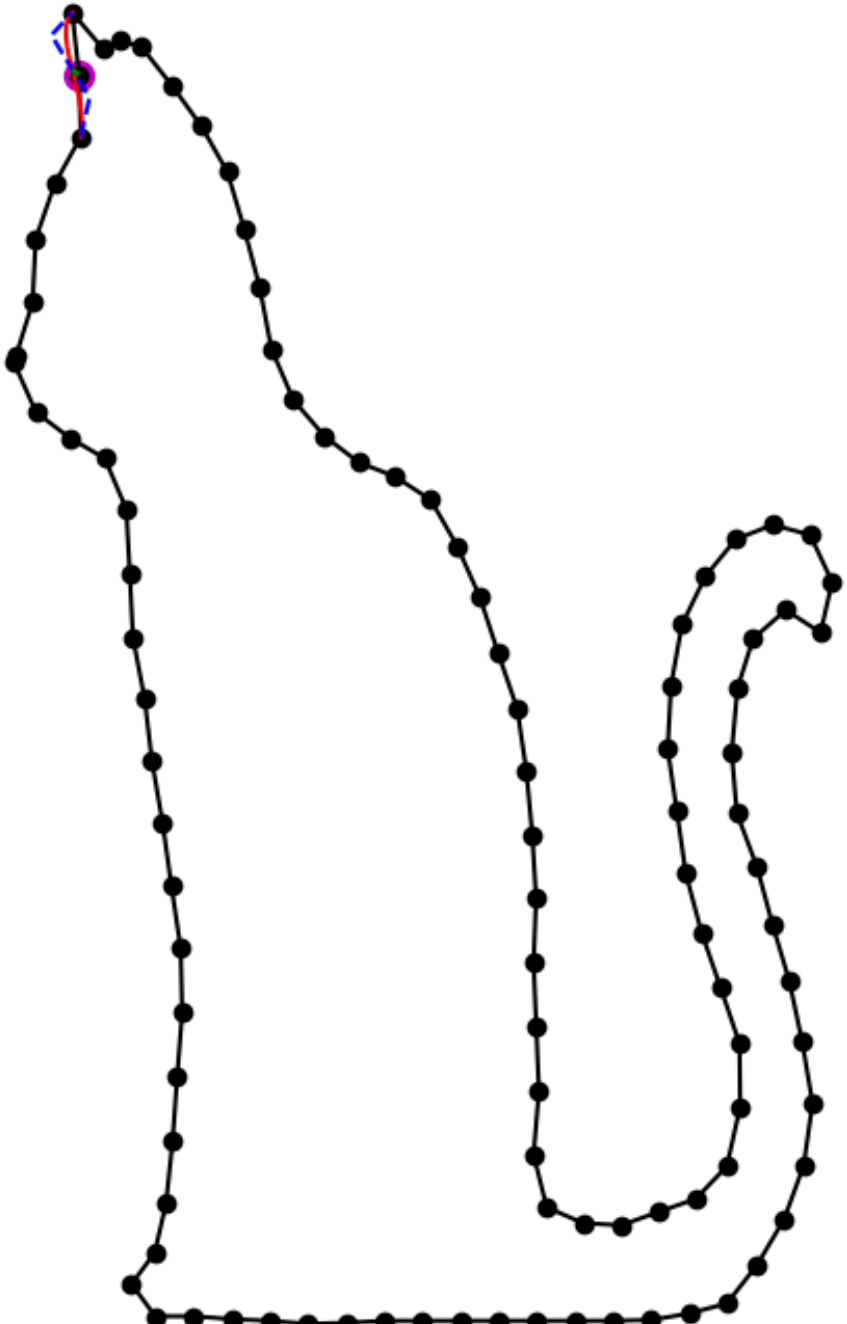




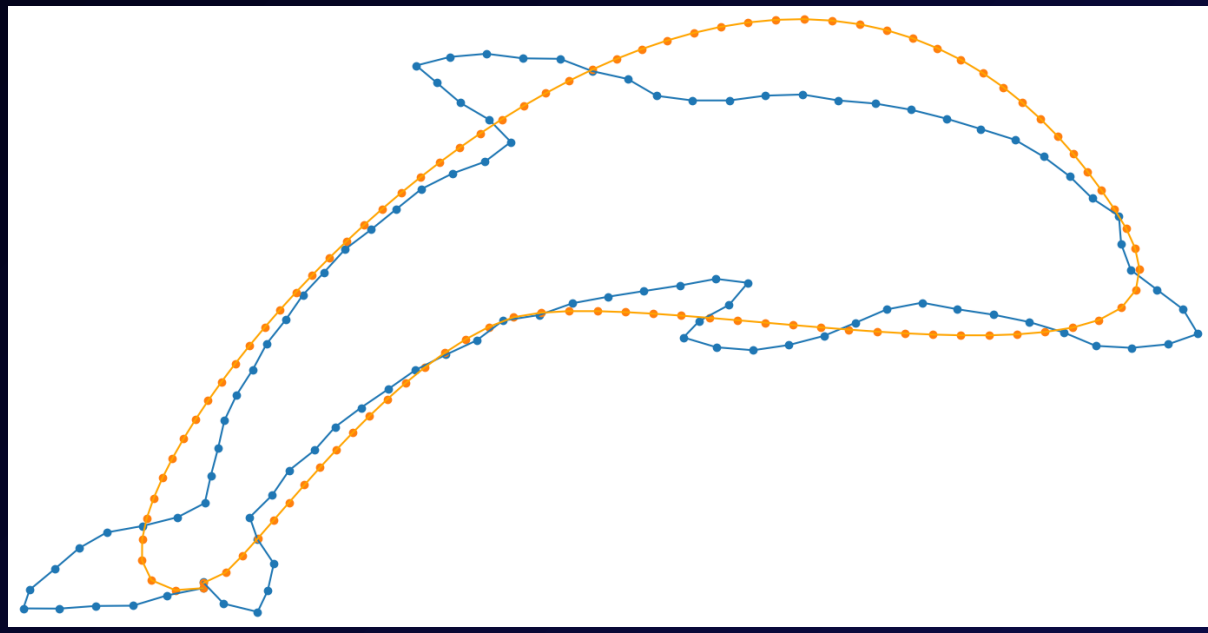




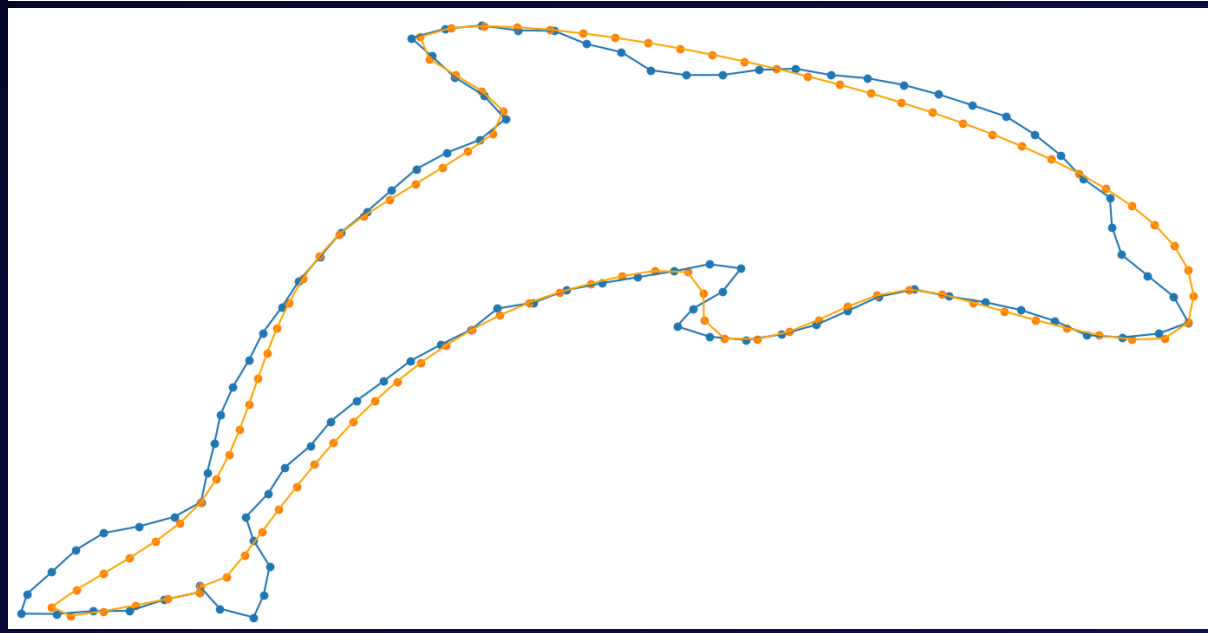




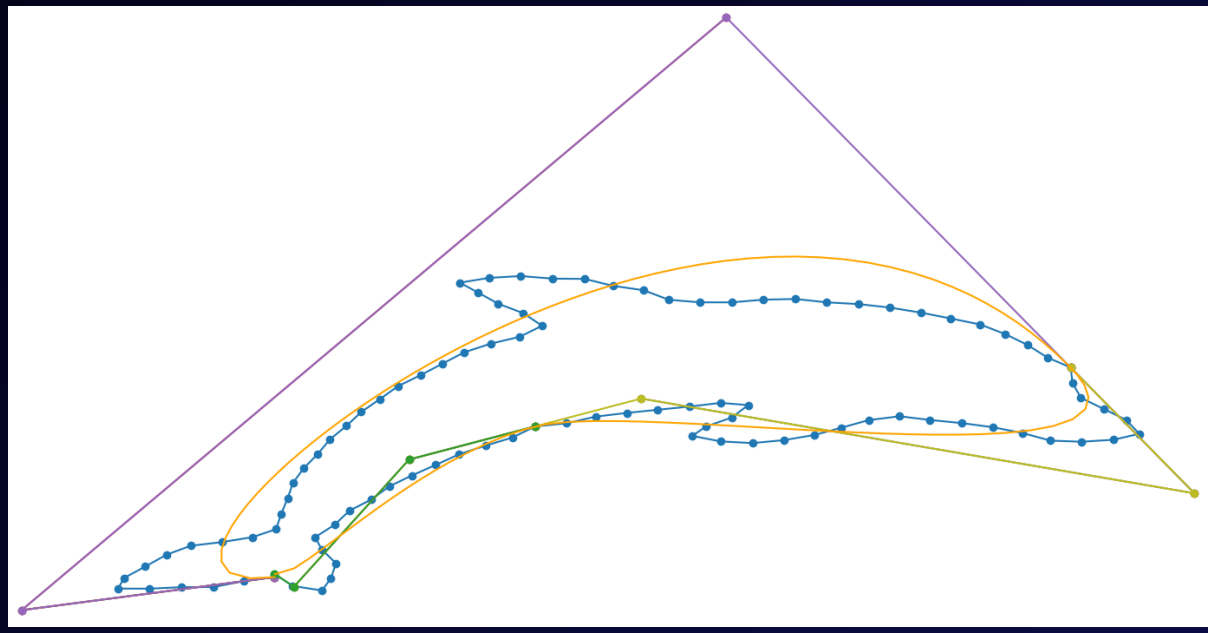
8 points de contrôle



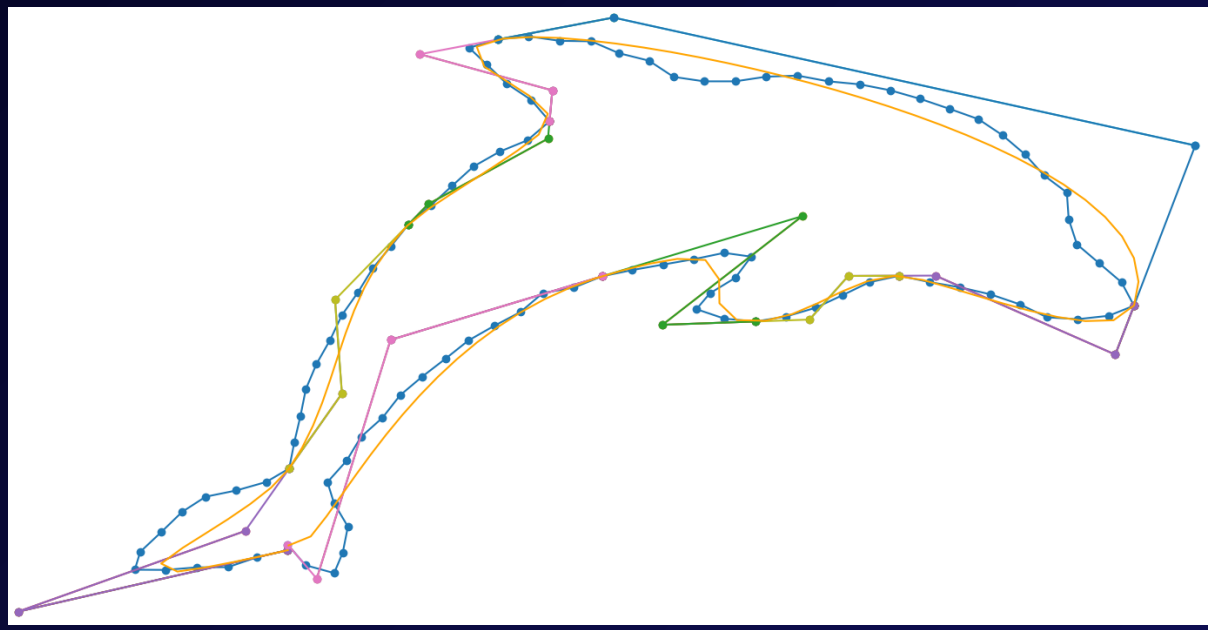
20 points de contrôle



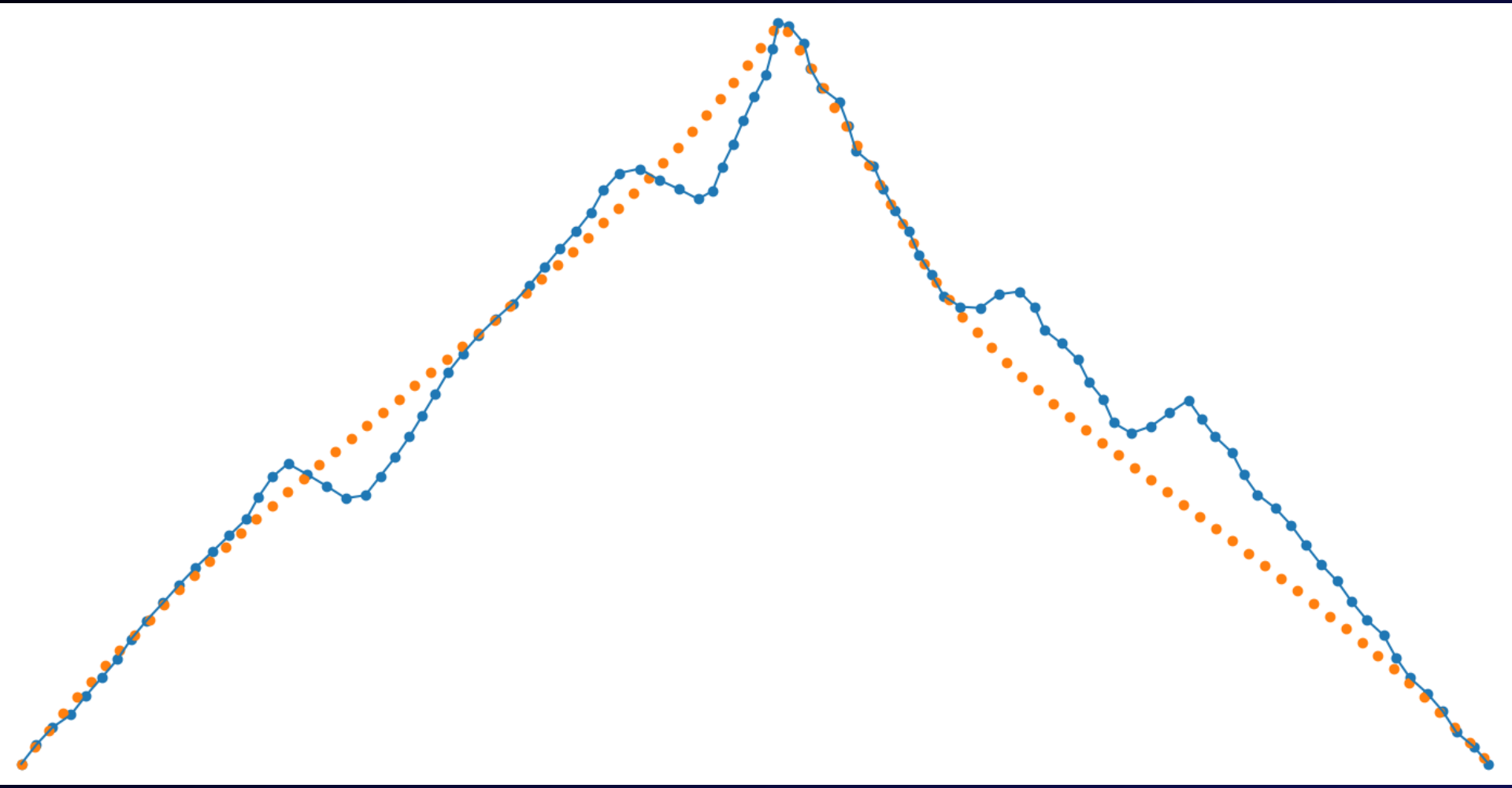
8 points de contrôle



20 points de contrôle



Courbe non fermés, 8 pts de contrôle, taux compression = 92 %



Évaluation des algorithmes

Quantité de données économisées :

taux de compression $\tau = 1 - \frac{taille\ obtenue}{taille\ initiale}$

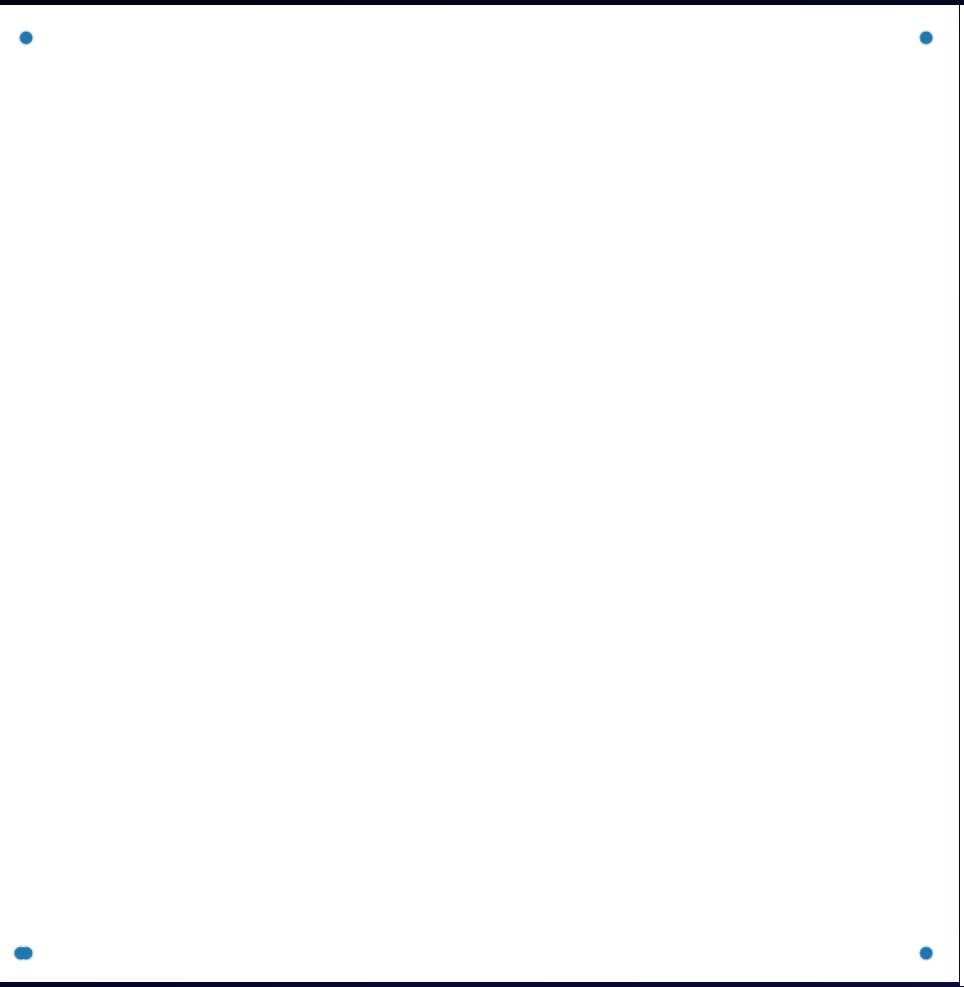
Qualité de courbe obtenue :

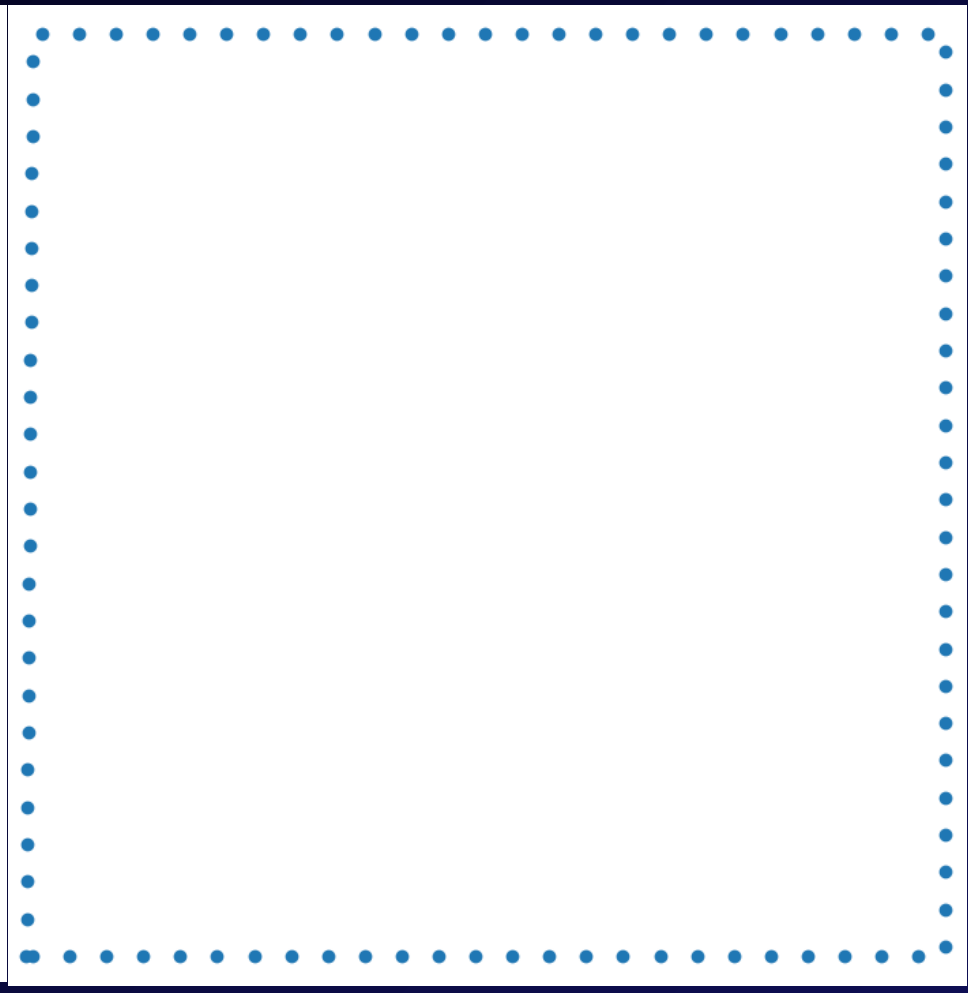
$(X, Y) \in (\mathbb{C}^n)^2$

distance = $||X - Y||_1$

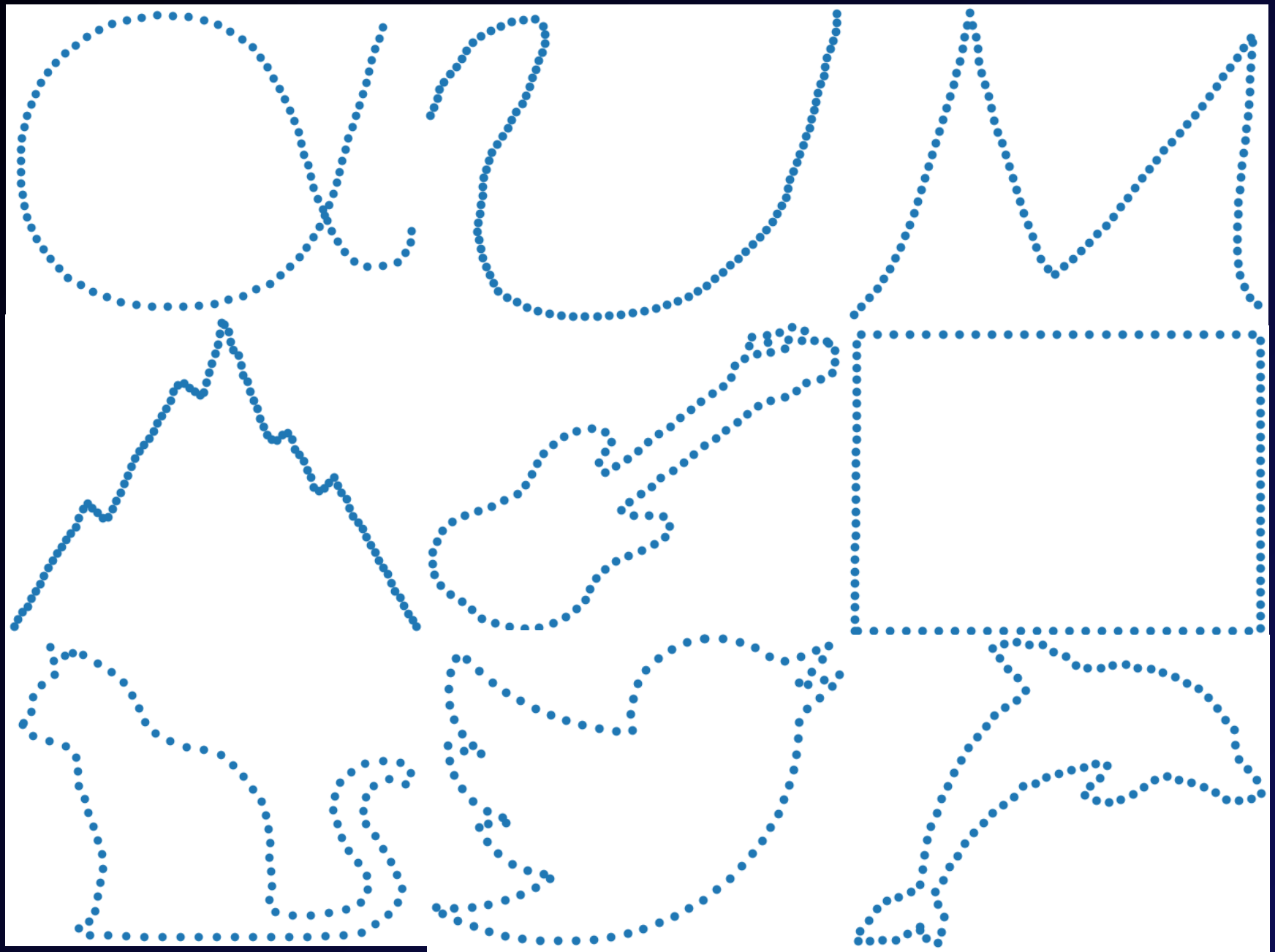
$$\forall x = (x_1, \ldots, x_n) \in \mathbb{C}^n, \|x\|_1 = \sum_{i=1}^{n} |x_i|$$

Avant reparamétrage Après reparamétrage





Évaluation des algorithmes



Évaluation des algorithmes

Taux de compression : 90 %

	Alpha	Lettre U	Lettre M	Montagne	Moyenne
Fourier	242%	562%	13%	337%	168%
TFD	160%	558%	3%	292%	142%
Schneider	0%	0%	0%	0%	0%

Évaluation des algorithmes

Taux de compression : 90 %

	Alpha	Lettre U	Lettre M	Montagne	Moyenne
Fourier	242%	562%	13%	337%	168%
TFD	160%	558%	3%	292%	142%
Schneider	0%	0%	0%	0%	0%

Taux de compression : 76 %

	Alpha	Lettre U	Lettre M	Montagne	Moyenne
Fourier	1281%	1045%	501%	527%	658%
TFD	662%	714%	337%	391%	446%
Schneider	0%	0%	0%	0%	0%

Taux de compression : 60 %

	Alpha	Lettre U	Lettre M	Montagne	Moyenne
Fourier	139%	1497%	430%	50%	72%
TFD	0%	802%	202%	0%	0%
Schneider	878%	0%	0%	26%	75%

Évaluation des algorithmes

Taux de compression : 90 %

	Guitare	Rectangle	Chat	Oiseau	Dauphin	Moyenne
Fourier	34.70%	518.01%	36.25%	27.23%	44.73%	62.60%
TFD	0.00%	0.00%	0.00%	0.00%	0.00%	0.00%
Schneider	227.24%	355.71%	143.37%	262.43%	366.57%	262.57%

Taux de compression : 76 %

	Guitare	Rectangle	Chat	Oiseau	Dauphin	Moyenne
Fourier	114%	1182%	225%	110%	132%	207%
TFD	0%	0%	0%	0%	0%	0%
Schneider	1087%	189%	153%	400%	210%	266%

Taux de compression : 60 %

	Guitare	Rectangle	Chat	Oiseau	Dauphin	Moyenne
Fourier	287%	2000%	467%	279%	393%	483%
TFD	0%	0%	0%	0%	0%	0%
Schneider	393%	308%	428%	204%	399%	328%

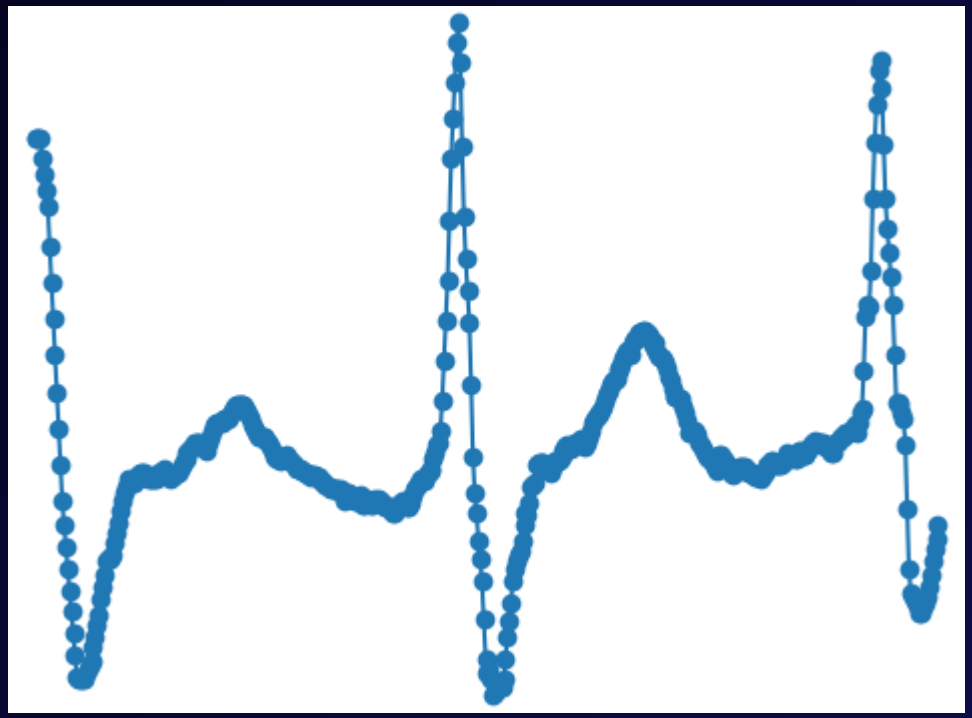
Application : Électrocardiogramme (ECG)

Base de donnée utilisée :

A large scale 12-lead electrocardiogram database for arrhythmia study (version 1.0.0) https://physionet.org/content/ecg-arrhythmia/1.0.0/. (CC BY 4.0)

Données utilisées :
Courbes de l'ECG des poitrines de 45,152 patients

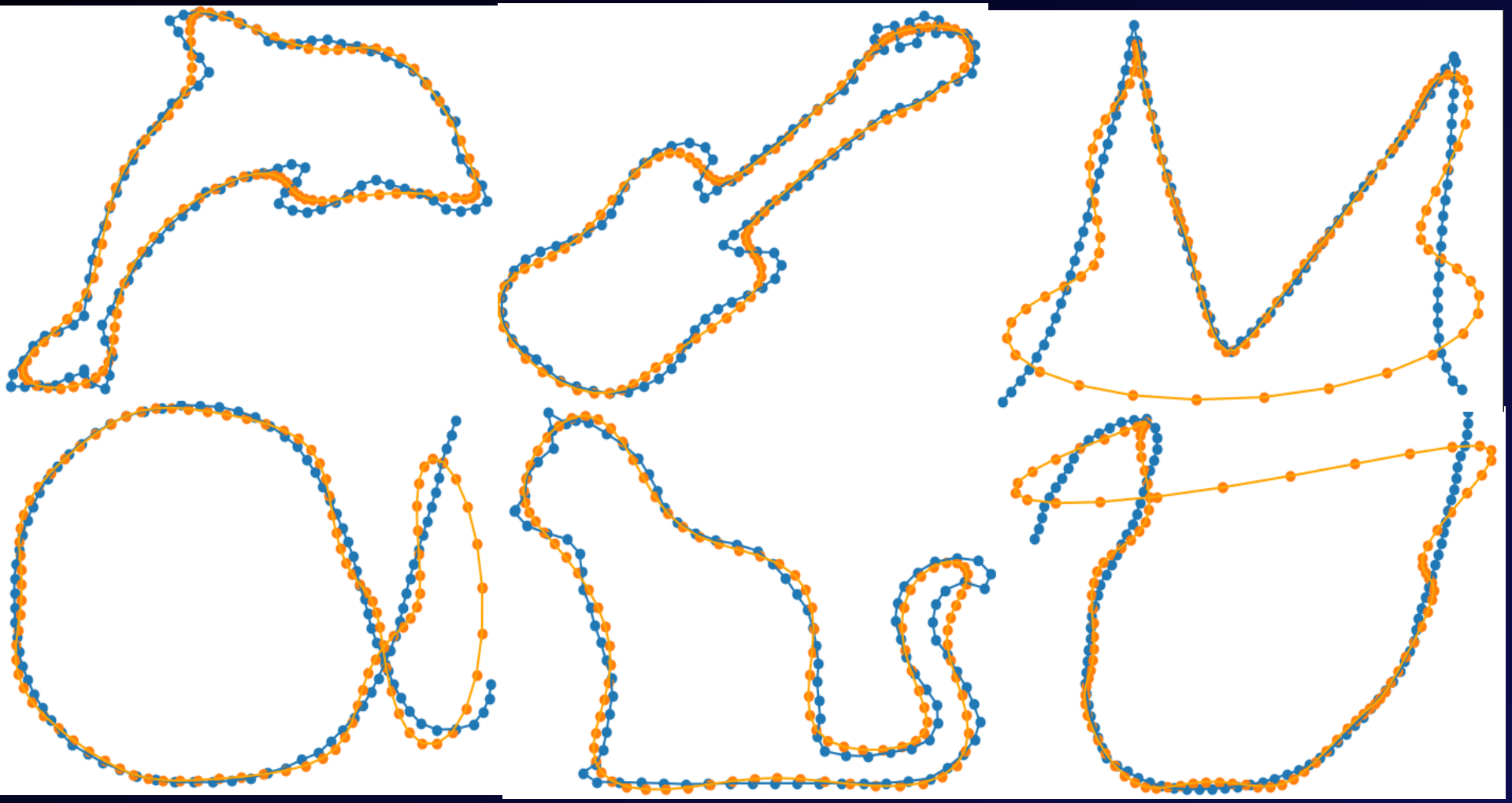
Exemple :



Application : Électrocardiogramme (ECG)

	τ=90%		τ=76%		τ=60%	
	Moyenne	Écart	Moyenne	Écart	Moyenne	Écart
Fourier	51.36	6%	16.35	41%	13.63	95%
TFD	48.38	0%	11.61	0%	6.99	0%
Schneider	712.62	1373%	664.20	5622%	634.33	8970%

Merci de m’avoir écouté !



Annexe

Taux de compression : 90 %

Image	Alpha	Carre	Chat	Dauphin	Guitare	Lettre U	Lettre M	Oiseau	Montagne	Moyenne
	1.49	0.77	0.95	1.00	0.91	2.38	2.43	0.93	4.22	1.49
Fourier	242%	518%	36%	45%	35%	562%	38%	27%	334%	204%
	1.13	0.12	0.70	0.69	0.67	2.36	2.22	0.73	3.79	1.13
TFD	160%	0%	0%	0%	0%	558%	26%	0%	290%	115%
	0.43	0.57	1.70	3.64	2.20	0.36	1.77	2.58	0.97	0.43
Schneider	0%	356%	143%	427%	227%	0%	0%	253%	0%	156%

Taux de compression : 66 %

Image	Alpha	Carre	Chat	Dauphin	Guitare	Lettre U	Lettre M	Oiseau	Montagne	Moyenne
	0.82	0.50	0.50	0.50	0.48	1.23	1.24	0.46	2.11	0.87
Fourier	1592%	1772%	343%	267%	237%	1349%	390%	182%	508%	738%
	0.38	0.03	0.11	0.14	0.14	0.76	0.84	0.16	1.47	0.45
TFD	684%	0%	0%	0%	0%	795%	233%	0%	325%	226%
	0.05	0.10	0.37	0.35	0.96	0.09	0.25	0.92	0.35	0.38
Schneider	0%	279%	229%	155%	579%	0%	0%	460%	0%	189%

Taux de compression : 50 %

Image	Alpha	Carre	Chat	Dauphin	Guitare	Lettre U	Lettre M	Oiseau	Montagne	Moyenne
	0.78	0.49	0.50	0.49	0.46	1.15	1.15	0.46	1.88	0.78
Fourier	153%	2597%	611%	562%	449%	1695%	388%	514%	56%	781%
	0.31	0.02	0.07	0.07	0.08	0.66	0.66	0.07	1.21	0.31
TFD	0%	0%	0%	0%	0%	934%	178%	0%	0%	124%
	3.23	0.10	0.41	0.34	0.43	0.06	0.24	0.21	1.65	3.23
Schneider	946%	425%	487%	362%	417%	0%	0%	181%	36%	317%

Annexe

Code d'explication comparaison courbes Schneider p.26

```python
import numpy as np
import matplotlib.pyplot as plt
def f(t):
    theta = np.pi * t
    return np.vstack((np.cos(theta), np.sin(theta))).T
def g(t): return np.vstack((1 - 2*t + t**2, 2*t*(1 - t))).T
t_cont = np.linspace(0, 1, 200)
f_cont,g_cont = f(t_cont), g(t_cont)
M = 10 # Points échantillonnés sur f
t_uniform = np.linspace(0, 1, M)
f_pts = f(t_uniform)
dists = [0.0] # Paramétrage par longueur de corde
for P, Q in zip(f_pts[:-1], f_pts[1:]): dists.append(dists[-1] + np.linalg.norm(Q - P))
u = [d / dists[-1] for d in dists]
g_pts = g(np.array(u))
errors = np.linalg.norm(f_pts - g_pts, axis=1) # Erreurs et erreur maximale
idx_max = int(np.argmax(errors))
plt.figure()
plt.plot(f_pts[:, 0], f_pts[:, 1], label='f (demi-cercle)', color='C0')
plt.plot(g_cont[:, 0], g_cont[:, 1], label='g (approximation)', color='C1')
plt.scatter(f_pts[:, 0], f_pts[:, 1], label='f(u_i)', color='C0')
plt.scatter(g_pts[:, 0], g_pts[:, 1], label='g(u_i)', marker='x', color='C1')
for i, pt in enumerate(f_pts):
    if i<5 and i != 0: plt.text(pt[0] + 0.02, pt[1] + 0.02, f'$f(t_{i})$', fontsize=20, color='black')
for i, pt in enumerate(g_pts):
    if i<5 and i !=0: plt.text(pt[0] + 0.02, pt[1] + 0.02, f'$g(t_{i})$', fontsize=20, color='black')
for i in range(len(u)): # Segments reliant f(u_i) et g(u_i)
    plt.plot([f_pts[i, 0], g_pts[i, 0]], [f_pts[i, 1], g_pts[i, 1]], color='gray', linestyle='--')
i = idx_max # Segment de l'erreur maximale en gras
plt.plot([f_pts[i, 0], g_pts[i, 0]], [f_pts[i, 1], g_pts[i, 1]], color='red', linewidth=2, label='erreur max')
plt.title("Erreur max entre f(u_i) et g(u_i)")
plt.axis('equal')
plt.grid(True)
plt.show()
```

A supprimer !